



Degree Project in Electrical Engineering, specialising in Embedded Systems

Second cycle, 30 credits

A Systolic Array-based General Convolutional Neural Network Accelerator

JINGWEI LI

A Systolic Array-based General Convolutional Neural Network Accelerator

JINGWEI LI

Master's Programme, Embedded Systems, 120 credits

Date: November 25, 2024

Supervisor: Wenyao Zhu

Examiner: Prof. Zhonghai Lu

School of Electrical Engineering and Computer Science

Swedish title: En systolisk array-baserad accelerator för generella
fälttningsneurala nätverk

Abstract

With the widespread application of Artificial Intelligence (AI), particularly the rapid development of Deep Neural Networks (DNNs), researchers are increasingly recognizing the critical role that an excellent hardware platform plays in facilitating the broad application and innovative research of DNNs. In embedded or mobile platforms, Field-Programmable Gate Arrays (FPGAs) have gradually attracted researchers' attention due to their flexibility, excellent energy efficiency, and cost-effectiveness. However, how to realize a unified architecture on FPGAs that efficiently supports acceleration on Convolutional Neural Network (CNN) inference remains a problem. In this thesis, we propose a Systolic Array (SA)-based general CNN hardware accelerator.

The accelerator is fully implemented in Verilog HDL. We use a large number of on-chip buffers to tailor the special data storage module to meet the parallel reading and writing requirements of the SA, reducing the number of accesses to external memory. During the execution of one layer, the accelerator can continuously perform data loading, computation, and storage processes in a pipelined manner, resulting in almost no idle cycles. It can process convolutional layers, pooling layers, and fully connected layers in CNNs respectively, and perform nonlinear activation operations on the results. It can also adapt to different CNN models according to the configuration information. The accelerator is implemented on the Xilinx XC7Z020, operating at a frequency of 90 MHz. We deploy the LeNet model on this accelerator to recognize handwritten images of the MINST dataset to validate its function and compare its performance with the advanced Graphic Processing Unit (GPU) platform. The results indicate that the time to process one image is slightly faster than the Tesla T4 GPU and demonstrate better energy efficiency. Furthermore, when compared with the embedded processor Cortex-A9, which uses the same CNN model and data quantization method, our design achieves a processing speed that is 192 times faster while consuming only one-third of its power. In conclusion, our accelerator has the ability to process different CNNs according to the configuration at deployment time and exhibits remarkable energy efficiency.

Keywords

Convolutional neural network, Hardware accelerator, Systolic array

Sammanfattning

Med den utbredda tillämpningen av artificiell intelligens (AI), och i synnerhet den snabba utvecklingen av djupa neurala nätverk (DNN), inser forskarna alltmer den avgörande roll som en utmärkt hårdvaruplattform spelar för att underlätta den breda tillämpningen och innovativa forskningen av DNN. I inbyggda eller mobila plattformar har Field-Programmable Gate Arrays (FPGA) gradvis dragit till sig forskarnas uppmärksamhet på grund av sin flexibilitet, utmärkta energieffektivitet och kostnadseffektivitet. Hur man realiserar en enhetlig arkitektur på FPGA: er som effektivt stöder acceleration av Convolutional Neural Network (CNN)-inferens förblir dock ett problem. I den här avhandlingen föreslår vi en Systolic Array (SA)-baserad generell accelerator för CNN.

Acceleratorn är helt implementerad i Verilog HDL. Vi använder ett stort antal buffertar på chippet för att skraddarsy den speciella datalagringsmodulen så att den uppfyller SA:s krav på parallell läsning och skrivning, vilket minskar antalet åtkomster till externt minne. Under exekveringen av ett lager kan acceleratorn kontinuerligt utföra dataladdnings-, beräknings- och lagringsprocesser i pipeline, vilket resulterar i nästan inga tomgångscyklar. Den kan bearbeta konvolutionslager, poolningslager respektive helt anslutna lager i CNN och utföra icke-linjära aktiveringsoperationer på resultaten. Den kan också anpassa sig till olika CNN-modeller enligt konfigurationsinformationen. Acceleratorn är implementerad på Xilinx XC7Z020 och arbetar med en frekvens på 90 MHz. Vi använder LeNet-modellen på denna accelerator för att känna igen handskrivna bilder från MINST-datasetet för att validera dess funktion och jämföra dess prestanda med den avancerade GPU-plattformen (Graphic Processing Unit). Resultaten visar att tiden det tar att bearbeta en bild är något kortare än för Tesla T4 GPU och att energieffektiviteten är bättre. Jämfört med den inbäddade processorn Cortex-A9, som använder samma CNN-modell och datakvantiseringsmetod, uppnår vår design dessutom en bearbetningshastighet som är 192 gånger snabbare samtidigt som den bara förbrukar en tredjedel av sin effekt. Sammanfattningsvis har vår accelerator förmågan att bearbeta olika CNN:er enligt konfigurationen vid driftsättningen och uppvisar en anmärkningsvärd energieffektivitet.

Nyckelord

Konvolutionellt neuronnätverk, Hårdvaruaccelerator, Systolisk matris

Acknowledgments

This thesis has taken a lot of effort from me at every step of the way, from literature research, to code implementation, to the final writing of the thesis. During this long process, it is thanks to the generous help of many people that I have been able to complete this thesis smoothly.

Here, I would like to express my sincere gratitude to Prof. Zhonghai Lu. At the beginning of the thesis, Prof. Lu provided me with crucial guidance on topic selection. He fully respected my interests, encouraged me to independently choose the thesis topic, and provided me with abundant and highly valuable literature in this field, laying a solid foundation for my research. During the code implementation stage, with the help of his guidance, I established the design ideas for each module. Besides, during the thesis revision process, the meticulous guidance of Prof. Lu has made my thesis more complete in structure and clearer in content. I would also like to thank Prof. Lu's two doctoral students, Wenyao Zhu and Yizhi Chen. During the communication with them, many technical problems that bothered me have been readily solved. It is especially worth mentioning that Wenyao Zhu proposed a large number of constructive suggestions for my thesis revision, making the thesis better.

I am also deeply grateful to my parents. Without their support, I would never have been able to cross the ocean to study in a foreign country, let alone complete my master's thesis report. Their love and support are the strongest backings on my educational journey in a foreign land. Finally, I want to thank my friends Quan Li, Chen Ling, and Haolin Liu. During my days in a foreign country, it was precisely because of their company that I was able to maintain an optimistic attitude when facing numerous difficulties in thesis writing.

Stockholm, November 2024
Jingwei Li

Contents

1	Introduction	1
1.1	Problem	3
1.2	Purpose	3
1.3	Goals	3
1.4	Ethics and sustainability	4
1.5	Delimitations	4
1.6	Structure of the thesis	5
2	Background	7
2.1	Convolutional neural network	8
2.2	Systolic array	10
2.2.1	Advantages of systolic array	10
2.2.2	Two-dimensional systolic array	13
2.2.2.1	Workload of the two-dimensional systolic array	14
2.2.2.2	Data flow of the two-dimensional systolic array	16
2.3	Related work	20
2.4	Summary	22
3	Design of the SA-based CNN accelerator	23
3.1	Overview of the SA-based CNN accelerator design	23
3.2	Design of Configurator	27
3.3	Design of Folding Controller	28
3.4	Design of Data Storage Module	30
3.4.1	Design of the input memory	30
3.4.2	Design of the weight memory	31
3.4.3	Design of the output memory	32
3.5	Design of Data Processing Module	33

3.5.1	Arithmetic module	34
3.5.2	Pooling module	35
3.5.3	Activation function module	36
3.6	Summary	36
4	Implementation	37
4.1	Implementation of Configurator and Folding Controller	37
4.2	Implementation of Data Storage Module	40
4.2.1	Hardware architecture of input memory and data reading process	40
4.2.2	Hardware architecture of weight memory and data reading process	42
4.2.3	Hardware architectures of output memory and data communication process	44
4.3	Implementation of Data Processing Module	46
4.3.1	Hardware architecture of arithmetic module	46
4.3.2	Hardware architecture of pooling module	50
4.3.3	Hardware implementation of activation function module	52
4.4	Summary	52
5	Results and Analysis	53
5.1	Testing dataset and data quantization	53
5.2	Function validation	56
5.3	Performance results	58
5.4	Summary	61
6	Conclusions and Future work	63
6.1	Conclusions	63
6.2	Limitations	64
6.3	Future work	64
	References	67

List of Figures

2.1	A simple convolution and pooling process	9
2.2	A basic weight stationary systolic array architecture	11
2.3	Calculation process of a fully connected layer with the corresponding matrix operations	14
2.4	Calculation process of a convolutional layer with the corresponding matrix operations	16
2.5	Output stationary dataflow pattern of a systolic array	17
2.6	Weight stationary dataflow pattern of a systolic array	19
3.1	Overview of the hardware architecture	25
3.2	Overview of the dataflow timeline	26
3.3	Process of the foldings	28
3.4	Sliding process on convolutional windows	29
3.5	Overview of the output memory	33
4.1	Hardware architecture of the input memory	41
4.2	Hardware architecture of the weight memory	42
4.3	Hardware architecture of the CNN output buffer	44
4.4	Hardware architecture of the FC output buffer	45
4.5	Hardware architecture of one processing unit in the systolic array	47
4.6	Output timing diagram of the systolic array	48
4.7	The critical situation of two sets of consecutive convolutional windows producing outputs simultaneously in the array	50
4.8	Operation process of the pooling units	52
5.1	LeNet model	55
5.2	Comparison of the dataset before and after binarization	55
5.3	Function validation results of our design	57

List of Tables

2.1	Explanations of the concepts	14
4.1	Parameters and definitions in Configurator	38
5.1	Parameters and the corresponding values in Configurator . . .	56
5.2	Hardware resource consumption	58
5.3	Hardware resource consumption of each module in the accelerator	58
5.4	Hardware resource consumption of each sub-module in Data Storage Module	59
5.5	Performance comparison with other hardware platforms	59
5.6	Power consumption	60

List of acronyms and abbreviations

ACT	Activation Function
AI	Artificial Intelligence
ASIC	Application Specific Integrated Circuit
BRAM	Block Random Access Memory
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CW	Convolutional Window
DNN	Deep Neural Network
FC	Full Connected
FMAP	Feature Map
FPGA	Field-Programmable Gate Array
FSM	Finite State Machine
GPU	Graph Processing Unit
GSMC	Group-Structure-Maintained Compression
INFH	Input Feature Map Height
INFMAP	Input Feature Map
INFW	Input Feature Map Width
IS	Input Stationary
KH	Kernel Height
KW	Kernel Width
LUT	Look-Up Table
MLP	Multi-Layer Perception
NVML	NVIDIA Management Library
OFH	Output Feature Map Height
OFMAP	Output Feature Map

OFMW	Output Feature Map Width
OS	Output Stationary
P	Padding
PE	Processing Element
PFMAP	Pooling Feature Map
POH	Pooling Feature Map Height
POW	Pooling Feature Map Width
RAM	Random Access Memory
ROM	Read Only Memory
SA	Systolic Array
SDD	Sparsity-supported Dynamic Dataflow
SMT	Simultaneous Multithreading
SMT-SA	Simultaneous Multithreading in Systolic Arrays
TPU	Tensor Processing Unit
WS	Weight Stationary

Chapter 1

Introduction

Convolutional Neural Network (CNN) as an efficient Deep Neural Network (DNN) [1] is widely used in computer vision, target detection, image processing, and some other fields. CNN have received a lot of attention since they were proposed [2] and have become a hot research topic nowadays. With the development of CNNs, the number of parameters and logical operations in the network increases exponentially. At the same time, with the end of Moore's law and Dennard scaling [3], traditional computer platforms face many performance bottlenecks. This makes the traditional serial computer platforms unable to meet the computing capability requirements of processing deep learning tasks [4]. Therefore designing an efficient hardware architecture that fits the computational pattern of neural networks well and has powerful computational capabilities has become an urgent requirement.

The emergence of neural network accelerators is of great significance for promoting the development of Artificial Intelligence (AI) technology. It can not only promote the widespread deployment of AI in practical applications but also can help develop new neural network models. Based on the using scenarios, we can simply divide the design direction of neural network accelerators into high-performance design and low-power design. For a high-performance neural network accelerator, it can significantly reduce the training and inference time, and reduce the system's computation time and response time. It is important in real-time systems. For a highly reliable low-power neural network accelerator, it can work stably for a long time in harsh environments under the basic performance requirements.

In order to solve this problem, researchers have also conducted extensive

exploration of different hardware platforms and architectures. Researchers tried to deploy neuron network models on **Graph Processing Units (GPUs)**. Compared with traditional computer platforms, such as **Central Processing Units (CPUs)**, which process data sequentially, **GPUs** can enable parallel computing of large amounts of data with a unique structure. This gives **GPUs** an irreplaceable advantage in the training and inference of large-scale neural networks. However, **GPUs** also have some obvious shortcomings and are not suitable for all cases. **GPUs** are not specifically designed for handling deep learning tasks, so only a part of the functions meets the actual demands of deep learning tasks. Secondly, **GPUs** have high power consumption which means they cannot adapt to a long-term low-power working environment. In recent years, more and more researchers have focused on implementing neural network accelerators on **Field-Programmable Gate Array (FPGA)** and **Application Specific Integrated Circuit (ASIC)** platforms [5]. On the one hand, researchers can design the specific hardware structures according to the **CNN** models on **FPGAs** and **ASICs**. On the other hand, they can achieve a higher energy efficiency ratio compared to **CPUs** and **GPUs**. Especially, **FPGAs** are preferred by researchers for their cost-effective, flexible, and programmable features compared to expensive **ASICs**. However, for **ASIC** and **FPGA** platforms, researchers need to carefully design the hardware architecture of the accelerator because the architecture directly affects the efficiency and performance of processing neural networks on these platforms [6].

Systolic Array (SA), as a highly parallel computational structure, provides an alternative method for implementing **CNN** hardware accelerators. In contrast to conventional architectures which use a few powerful processors sequentially [7], the **SA** consists of multiple arithmetic units, allowing the parallel computation of multiple data at the same time. Moreover, these computational units are simple, highly reliable, cost-effective, and can be arranged according to the practice task requirements. When the **SA** works, the data only needs to be input into the array from different directions in each cycle. This relatively fixed data flow pattern makes it unnecessary to introduce complex control logic in **SA**, which greatly improves the stability of the system. At the same time, a large amount of data will be reused in the array, which means the data taken out from one access to the memory can be involved in multiple computational units. This design can significantly improve the throughput of the system under limited bandwidths [7]. In summary, the **SA** is well-suited for processing **CNNs**.

1.1 Problem

Based on the research and understanding of the literature, we can summarise the research points in four research questions:

1. How to design an efficient arithmetic module based on the **SA** that can support calculation operations in **CNNs**?
2. How to design an efficient data storage module that can meet the parallel data reading and writing requirements of the **SA**?
3. How to design an efficient data flow pattern to improve the performance of the accelerator?
4. How to design a data quantization method to be suitable for the operations on the **FPGA**?

1.2 Purpose

In this thesis, our purpose is to design an **SA**-based general **CNN** accelerator architecture. It can not only be verified in the simulation but also can be deployed on the **FPGA** and its functionality can be verified by the real neural network model. The **CNN** hardware accelerator can adjust the size of the **SA** within a certain range, and it can support convolutional operations with multiple channels, different strides, and different sizes of **Input Feature Maps (INFMAPs)**, as well as fully connected operations. It can also support maximum pooling operations and activation functions under certain conditions.

1.3 Goals

The main goal of this project is to design an **SA**-based **CNN** hardware accelerator that can adapt different **CNN** models within a certain range by adjusting the parameters and can be deployed on the **FPGA** for actual operation. This can be divided into the following four sub-goals:

1. Design an efficient arithmetic module based on the **SA** that can support the convolutional operations and fully connected operations in the **CNN** model simultaneously.

2. Design an efficient data storage module to store the large parameters of the **CNN** model and be able to meet the parallel data reading and writing requirements of the **SA**.
3. Design an efficient data flow to minimize the idle cycles during the operation of the accelerator as much as possible, thereby improving the performance of the accelerator.
4. Implement an appropriate data quantization method that can reduce the storage area and computational operations on the **FPGA** while ensuring less loss of precision.

1.4 Ethics and sustainability

In the project, we use random data in the simulation stage and the open-source MNIST [2] dataset in the hardware implementation stage to test our design. There are no issues such as user privacy data leakage during the testing process, and it can ensure that the inferences of the accelerator during the testing process are fair and unbiased.

Regarding sustainability, considering that performance requirements are the first priority of this design, this design uses a relatively large amount of hardware resources to improve system performance. However, the **SA** structure is usually composed of many simple computing units, and the corresponding control logic is also straightforward. This structure makes the accelerator have high reliability and can operate stably for a long time with a relatively high service life. At the same time, our accelerator has good energy efficiency and can effectively save energy, meeting the requirements of green environmental protection.

1.5 Delimitations

The design of the **SA**-based accelerator does not support arbitrary adjustment of the array size. Its dimensions can only be conditionally modified based on the **INFMAPs**, and the accelerator primarily supports square-shaped arrays. Additionally, the accelerator can only deploy certain patterns of **CNN** models. For example, the accelerator does not allow padding during computation and the pooling layer only supports max-pooling where the pooling window size is equal to the pooling stride.

Furthermore, a comprehensive hardware design typically requires the development of a high-coverage automated hardware testing platform to ensure that the design functions correctly under various conditions. However, the design and testing of hardware are often carried out by separate teams, and building a high-quality testing platform is as challenging as designing the hardware circuit itself, requiring specialized expertise in testing methodologies. Consequently, the development of an automated hardware testing platform falls outside the scope of this project. In this thesis, we will deploy our accelerator on Xilinx XC7Z020 FPGA to process a real **CNN** model and compare the computation results obtained from the **FPGA** with the results from Python using several representative test cases to verify the correctness of the design.

1.6 Structure of the thesis

Chapter 2 presents relevant background information about **CNN** and **SA**. Chapter 3 introduces the general design concept of the accelerator and explains the specific working mechanism of each module, including Configurator, Folding Controller, Data Storage Module, and Data Processing Module. In Chapter 4, we explain the hardware architecture and implementation of this **CNN** accelerator. Chapter 5 reports the experimental results of the project and compares the performance of our design to other advanced hardware platforms. Chapter 6 concludes the project and presents a plan for future work.

Chapter 2

Background

CNN is a type of **DNN** and is widely used in computer vision problems, such as object detection, image classification, and other areas. Specifically, **CNN** will use a convolutional layer to extract features from the input image, a pooling layer to reduce the feature dimensions, and a fully connected layer to make a final prediction. The **CNN** learns useful knowledge from a large amount of training data and utilizes gradient descent and backpropagation to obtain the optimal convolutional kernel. This structure makes **CNN** highly efficient and accurate in processing images.

With the rapid development of **CNN** over these years, researchers have endowed **CNN** with the ability to handle increasingly complex computer vision problems through meticulous design. However, the enhancement of its problem-solving capabilities has directly led to a more complex structure and a significant increase in its overall scale. In particular, the number of logic operations within **CNN** has increased dramatically with the growth in parameters. How to quickly and efficiently implement the training of **CNN** models, as well as utilize existing **CNN** models for forward inference, has become a hot research topic.

In previous studies, researchers often utilized **CPUs** or **GPUs** to implement **CNN** models. Although both **CPUs** and **GPUs** possess notable advantages in this domain, particularly **GPUs**, which are indispensable for implementing large-scale or high-performance **CNN** models, these computing units were not specifically designed for deep learning tasks. Consequently, only a small part of their functions is well-suited for deep machine learning [8], and they hard to meet the low-power consumption requirements of mobile devices.

Recently, there has been a growing interest in utilizing **ASICs** or **FPGAs** to develop specialized hardware architectures for specific **CNN** models. **ASICs** and **FPGAs** can be designed to closely align with the computational processes of specific neural networks, thereby achieving higher energy efficiency compared to **CPUs** and **GPUs**.

While **ASIC** offers superior performance and power efficiency due to its higher clock frequencies and better power management, it has significantly higher development costs. In contrast, **FPGA** provides a cost-effective, flexible, and easily implementable solution for accelerating **CNN** algorithms, making it become an attractive choice [9].

2.1 Convolutional neural network

CNN evolved from **Multi-Layer Perception (MLP)**, inheriting its strengths while introducing improvements that significantly enhance network performance and reduce the number of weights in the network model [2]. Specifically, **MLP** flattens the input image into a vector and fully connects the neurons between adjacent layers via weights. This approach results in the loss of the spatial information of the original image and introduces a large number of weights. In contrast, **CNN** uses weight sharing and local connectivity. These modifications result in a more efficient use of parameters, thereby reducing the overall complexity of the network model.

CNN exhibits a variety of structures, but generally, a **CNN** model consists of multiple convolutional layers, pooling layers inserted between adjacent convolutional layers, activation functions, and several fully connected layers at the end. Specifically, the convolutional layers perform convolutions on the **Feature Maps (FMAPs)** obtained from the preceding layer using convolutional kernels to extract features. The outputs from the convolutional layers are then fed into the appropriate activation functions, which can introduce non-linearities, allowing the network to learn and represent more complex patterns. The results of the activation functions are then passed to the pooling layers for down-sampling, which can further reduce the network parameters.

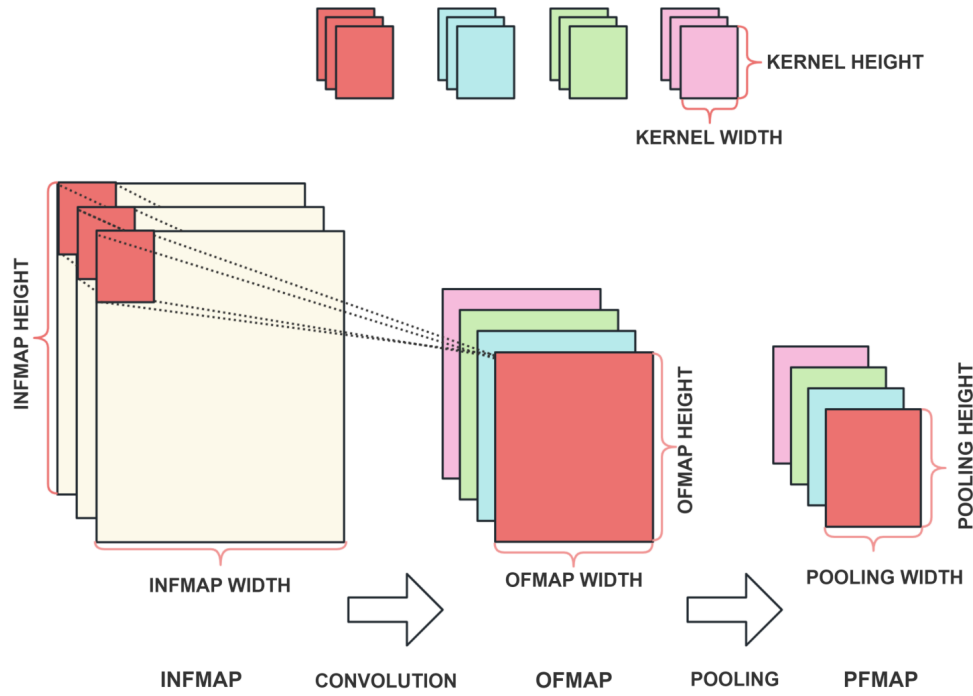


Figure 2.1: A simple convolution and pooling process

Figure 2.1 shows a simple convolution and pooling process. We define the left feature map as the **INMAP**. The feature map obtained after the convolutional operations of the **INMAP** as the **Output Feature Map (OFMAP)**, and the feature map after the pooling operations of **OFMAP** as the **Pooling Feature Map (PFMAP)**. The **PFMAP** of the previous layer will be used as the **INMAP** of the next layer. The convolutional layer receives an **INMAP** with three input channels, an **Input Feature Map Height (INFH)** and an **Input Feature Map Width (INFW)**. It performs convolutional operations using four different kernels, resulting in an **OFMAP** with four channels. **Output Feature Map Height (OFH)** and **Output Feature Map Width (OFMW)** can be calculated using Equation 2.1 and 2.2. **Padding (P)** represents the number of zeros padded around the boundary of **INMAP** to ensure dimensions compatibility with the convolutional kernels. **Stride** denotes the distance the **Convolutional Window (CW)** slides each time. **Kernel Height (KH)** and **Kernel Width (KW)** denote the height and width dimensions of the convolutional kernels in the current convolutional layer, respectively.

$$OFH = \frac{(INFH + 2 \cdot P - KH)}{\text{Stride}} + 1 \quad (2.1)$$

$$\text{OFW} = \frac{(\text{INFW} + 2 \cdot \text{P} - \text{KW})}{\text{Stride}} + 1 \quad (2.2)$$

Further, the **OFMAP** is downsampled by a pooling layer, producing a feature map with four channels, a **Pooling Feature Map Height (POH)** and a **Pooling Feature Map Width (POW)**. This reduces the number of parameters in the network.

2.2 Systolic array

In the development history of **CNN** hardware accelerators, various hardware architectures have been tried and explored. **SA**, which supports high parallel operations and data reuse, aligns well with the computational patterns of convolution. This structure provides an alternative solution for implementing convolution units in **CNN** hardware accelerators. H.T. Kung summarized previous research experiences and first introduced the concept of **SA** in 1978 [7]. However, at the time, this architecture did not garner significant attention from researchers, and after early explorations, most researchers shifted their focus back to **GPUs** or multi-core processors [10].

In recent years, the resurgence of **AI** and deep machine learning has breathed new life into **SA**. Especially, the success of Google's **Tensor Processing Unit (TPU)** [4] demonstrated the feasibility of using an **SA** as the core to build the **CNN** hardware accelerator's convolution units and highlighted the powerful capability of **SA** in handling similar computational tasks. The **TPU**'s core utilizes an **SA** to optimize matrix multiplication. Data flows through an array of **Processing Elements (PEs)** after being fetched from memory. The data retrieved in one I/O operation can be reused across multiple **PEs**, and different **PEs** can perform parallel computations on distinct data. This method significantly reduces the required I/O operations for performing one convolutional layer, leading to improved computational efficiency and better utilization of resources. After the success of **TPU**, more and more **SA**-based neural network accelerators [11] [12] and neural network optimization methods serving **SAs** [13] [14] have been released.

2.2.1 Advantages of systolic array

SA is a computing architecture designed to efficiently handle computationally intensive tasks by leveraging data reuse and parallelism. The mechanism of **SA** is analogous to the human circulatory system. Just as blood is pumped

out by the heart and flows through different organs, participating in various physiological functions before returning to the heart, the data retrieved from memory in a single I/O operation to be streamed through multiple **PEs** for multiple computations, and subsequently storing these computation results back into memory.

Figure 2.2 demonstrates a basic weight stationary **SA** structure and the working mechanism of the **PEs** that compose the array. This architecture is primarily used to perform the computation described by Equation 2.3. For this **SA**, each **PE** contains a register and a multiplier. When the **SA** begins its operation, each **PE**'s register is preloaded with a weight. Subsequently, one input data x_i will be transported into the array for each cycle. When the first input data x_0 enters the **PE** storing w_0 , the computation begins. At this point, each **PE** performs a multiplication operation, calculating the product of the input and the weight, and passes this result to the adder. The adder sums all the multiplication results, yielding the output result y_0 for the current cycle.

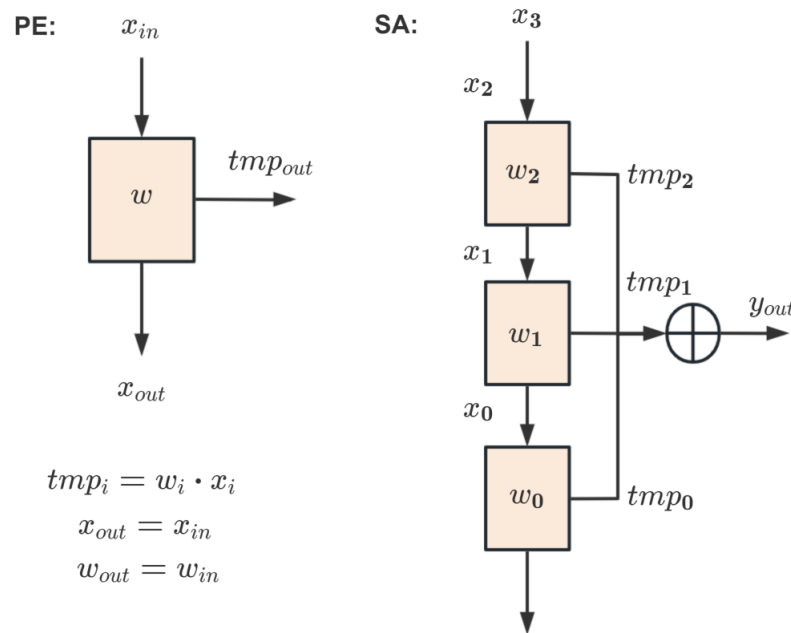


Figure 2.2: A basic weight stationary systolic array architecture

In the next cycle, x_0 exits the array, x_3 enters the array, x_1 and x_2 are passed forward to the preceding **PEs**, and the multiplication and addition operations are repeated to obtain y_1 . Subsequent outputs are calculated in a similar

manner. This is a just simple example to present the mechanism of the SA. In practice, researchers can customize different PEs and data flow patterns to form specific SA according to specific task requirements.

$$y_i = w_0 \cdot x_i + w_1 \cdot x_{i+1} + \dots + w_n \cdot x_{i+n} \quad (2.3)$$

From the above example, we can summarize four general advantages of SA as follows:

- **Good scalability, reliability and cost-effectiveness.** In this example, the SA is composed of multiple PEs and an adder connecting these PEs. Each PE can be considered to contain a register for preloading weight, a multiplier, and a bypass for passing input data. This relatively simple structure also makes the data and control flow pattern straightforward. The data only needs to enter the array serially from the top and proceed to the next PE in each cycle, which provides good reliability for the whole system. Furthermore, this modular design provides excellent scalability for the entire systolic system. When using SA to handle complex tasks, we can place multiple cells on a chip for parallel data operations to meet the performance requirements. This necessitates that each unit must be simple, efficient, and low-cost. The PE in SA can meet these requirements, which typically contain only a high-performance arithmetic unit and minimal memory space.
- **High degree of data parallel processing.** SA uses multiple units capable of parallel processing data instead of relying on a few powerful core processors in conventional architectures to sequentially handle data [7]. Specifically, in this example, once in the calculation stage, there are three PEs performing multiplication operations simultaneously in each cycle, obtaining the multiplication results. Although the current degree of data parallelism in the SA seems limited, once we scale the array up, especially extending it to a multi-dimensional SA, the amount of data parallel operations in the system will significantly increase. Moreover, through proper pipelining design, we can overlap some of the I/O operation times with the data computation times, further enhancing the performance and parallelism of the entire system.
- **Data reuse.** This weight stationary SA structure also exhibits significant data reuse. This is mainly reflected in the reuse of weights that are preloaded into the PEs as well as the input data. Once the weights are preloaded, these weights can be repeatedly used as new

input data continuously enters the array. Meanwhile, the input data read from the memory at one time is continuously fed to multiple PEs for multiplication operation. All input data other than x_0 are reused multiple times in this array. This approach can significantly enhance the throughput for outside communication and improve the performance of the system with relatively limited I/O bandwidths [7].

- **Matching compute rate to I/O bandwidth.** In SA, data is continuously passed between PEs, but only the PEs at the boundaries communicate with external memory for I/O operations to acquire data. This means only the top PE directly interacts with the external memory storing input data, while the rest of the PEs do not access the memory directly. This reduces the bandwidth demand of the entire system. Meanwhile, the modular design of SA allows the array structure to be easily adjusted to adapt to the I/O requirements of different tasks [7]. By adjusting the number and configuration of PEs, the system can be optimized to match different computation rates and I/O bandwidths.

Overall, the four main advantages of SA align well with the characteristics of convolution. The modular design not only reduces the system's cost and increases its reliability but also allows for customizing the SA according to different convolutional operation requirements. High data parallelism enables the array to perform multiple data computations simultaneously within a single cycle, which enhances the performance of the system. High data reuse means the array can achieve high data throughput without frequently accessing external memory. These advantages make it possible to build CNN hardware accelerators with SA.

However, a simple linear array can not meet the extensive and rapid data computation requirements of CNNs. Therefore, we further extend the system to a two-dimensional SA to address the challenges in convolutional operations.

2.2.2 Two-dimensional systolic array

Scaling a one-dimensional SA to a two-dimensional SA is not just a change from a linear array to a rectangular array. Data reuse and parallelism are both increased. In this section, we will focus on the structure and data flow patterns of two-dimensional SAs which are commonly used to implement CNN accelerators.

Table 2.1: Explanations of the concepts

Parameters	Definition	Label
SR	Rows of a systolic array	
SC	Columns of a systolic array	
CWN	Number of convolutional windows	0
CWS	Number of elements in a convolutional window	2
KN	Number of convolutional kernels	1
KS	Number of elements in a convolutional kernel	2
OCHN	Number of output channels	1
OCHS	Number of elements in an output channel	0

Table 2.1 lists some of the parameters that will be used below. Parameters with the same numeric label can be considered numerically equivalent.

2.2.2.1 Workload of the two-dimensional systolic array

SA can be directly applied to a **CNN** accelerator to complete the calculations of convolutional layers and fully connected layers, and significantly improve the calculation speed and data throughput.

$$O_m = I_0 \cdot w_0^m + I_1 \cdot w_1^m + \dots + I_n \cdot w_n^m \quad m \in [0, M), \quad n \in [0, N) \quad (2.4)$$

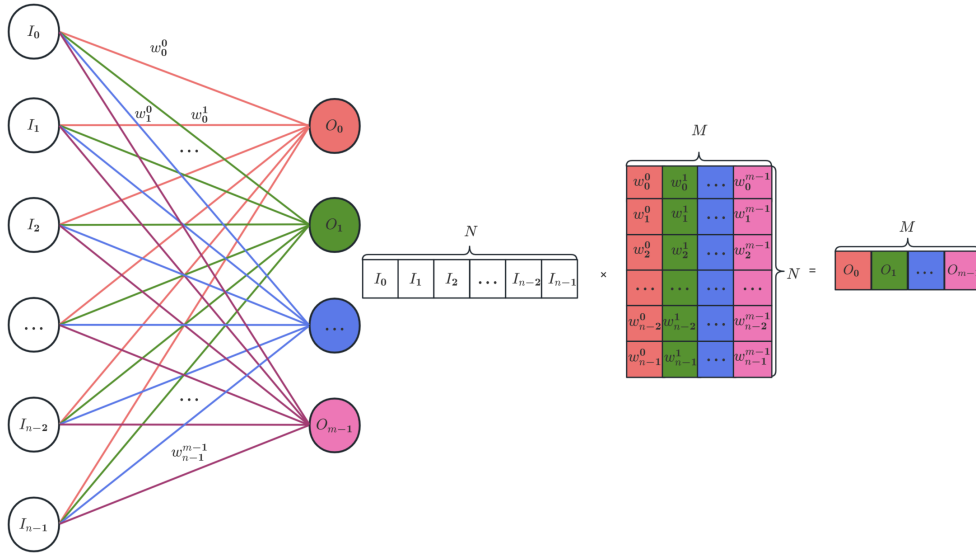


Figure 2.3: Calculation process of a fully connected layer with the corresponding matrix operations

Figure 2.3 shows the calculation process of a fully connected layer with a batch size of one, N input data, and M output data, as well as the corresponding matrix operation process. It can be simply regarded as a multiplication operation between a $1 \times N$ vector and an $N \times M$ matrix, and get a $1 \times M$ output vector. Equation 2.4 explains in detail the calculation formula of the output, where n represents the n^{th} input data, and m represents the m^{th} output data.

The operations of the convolutional layers can be regarded as an extension of the operations of the fully connected layers. It still follows the computational method of the fully connected layers. Nevertheless, the input of a convolutional layer is not merely a one-dimensional vector but a **FMAP** with multiple channels. We need to perform additional operations to process the input data of the convolutional layer.

Convolutional operation means that a **CW** of the same size as the convolutional kernel keeps sliding over the **INFMAP** according to the stride, and takes out the matrix. This process continues until the **CW** completely covers the whole **INFMAP**. Each element of the matrix is multiplied by the element at the corresponding position of the convolutional kernel. After completing all the multiplication operations, the result is summed up to get a pixel point in the corresponding output channel of the **OFMAP**.

Figure 2.4 illustrates the detailed process of convolving a three channel 5×5 **INFMAP** with four 3×3 convolutional kernels under a stride of one. We also set the batch size to one for a simpler illustration of this process. For this convolutional process, we can convert it into a general matrix multiplication of a 9×27 input matrix and a 27×4 weight matrix, and finally obtain a 9×4 output matrix. Specifically, 9 represents CWN and OCHS, 27 represents CWS and KS, and 4 represents KN and OCHN. We can summarise the unrolling convolution as the matrix multiplication of a $CWN \times KS$ matrix and a $KS \times KN$ matrix.

After unrolling the convolutional operation, we find that it is the same as the previous fully connected layer calculation process, except that it is extended in dimensions. Both of them can be generalized as general matrix multiplication. Therefore, we can use the two-dimensional **SA** to process the convolutional layers and the fully connected layers in **CNN** in the same way.

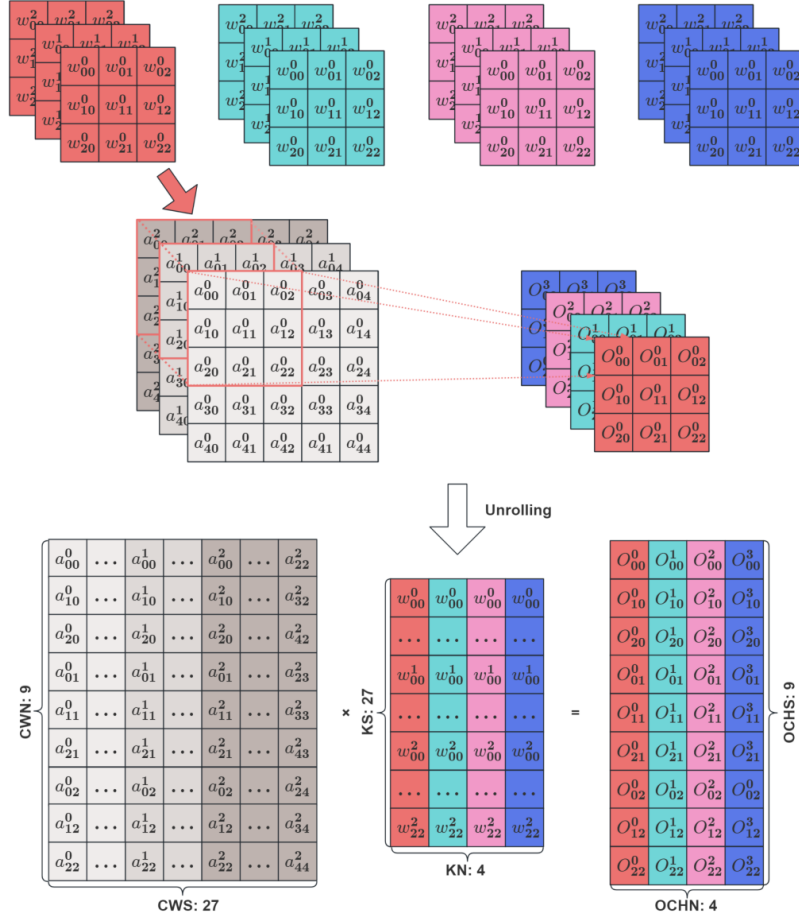


Figure 2.4: Calculation process of a convolutional layer with the corresponding matrix operations

2.2.2.2 Data flow of the two-dimensional systolic array

For **SA**, there are three typical data flow patterns, namely **Output Stationary (OS)**, **Weight Stationary (WS)**, and **Input Stationary (IS)** [15] [16] [17]. In this section, we will focus on the first two data flow patterns to explain how data are mapped into **SA**, as well as data reuse and the ideal number of time cycles during the process.

OS is a data flow pattern in which the outputs are stationary and the inputs and weights are continuously injected into the **SA** from two directions respectively. In Figure 2.5, we use 16 **PEs** to form a 4×4 **SA**. Each row of the **SA** receives all elements of a **CW**, each column receives all elements of a kernel, and the

outputs are fixed in the **PEs**. For each cycle, the inputs and the weights are pushed forward once. Whenever an input and a weight enter a **PE**, they are multiplied. After that, the multiplication result is added to the partial sum of the previous cycle to get the new partial sum. This operation is repeated for each subsequent cycle until the **PE** completely receives all the elements of a **CW**. At this time, the partial sum is the final output and flows out from the bottom of the **SA**. It is worth noting that the data input of each row of **SA** lags one cycle behind the previous row. The columns of **SA** also conform to this pattern.

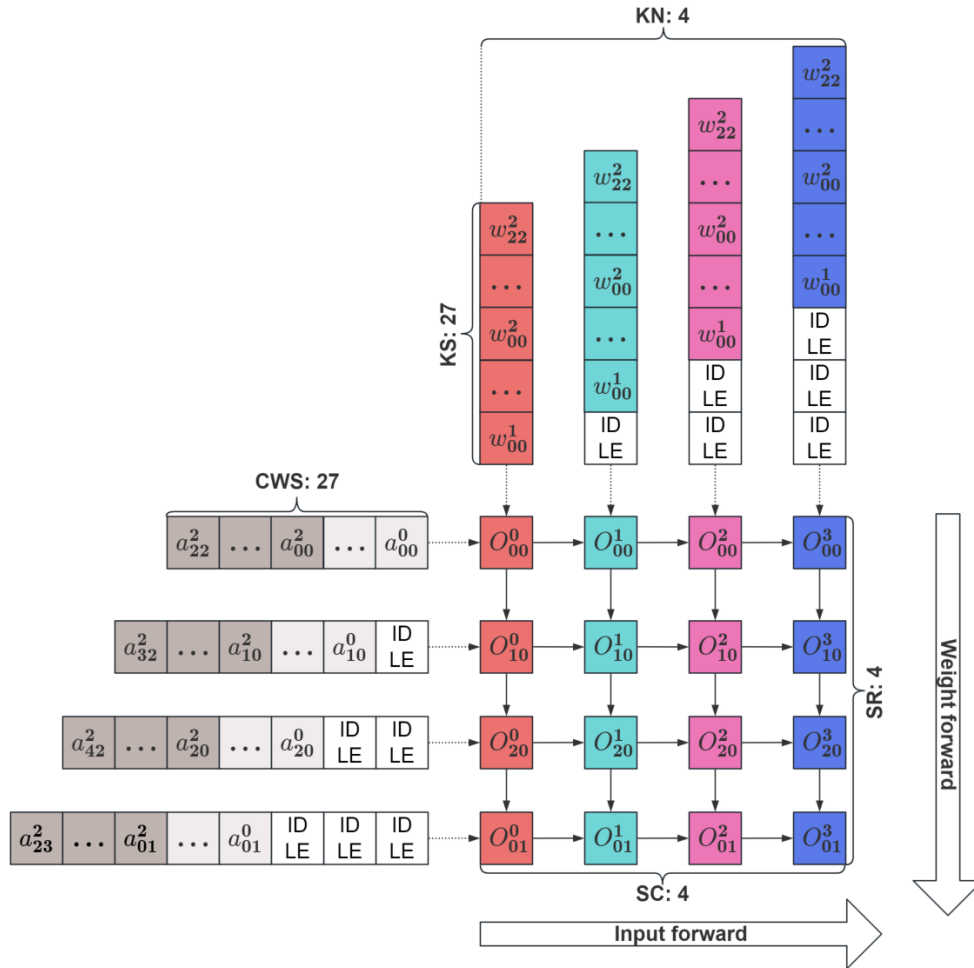


Figure 2.5: Output stationary dataflow pattern of a systolic array

For an $SR \times SC$ **OS SA**, a complete operation can obtain $SR \times SC$ elements in **OFMAP**, which means there are SC output channels with SR elements in

each output channel. Therefore, in Figure 2.5, 16 elements can be calculated, which are 4 elements in each of the 4 output channels in the OFMAP. However, according to the convolutional operation shown in Figure 2.4, there are 4 output channels and each output channel has 9 elements. Obviously, the array in Figure 2.5 can not obtain the complete OFMAP in one operation. In order to complete the entire convolutional operation, we can use more hardware resources to expand the original SA, or we can use a relatively limited array multiple times. The latter method seems to be a better one. This is because the input of the convolution can be easily increased, but the hardware resources on the chip are always limited. Therefore, this thesis will focus on how to use an SA repeatedly to complete all convolutional operations in a large INFMAP in combination with hardware design in the subsequent chapters.

Data reuse mainly occurs in two aspects:

1. One CW propagated to the right is able to be used by multiple convolutional kernels, generating one element in different output channels respectively.
2. One convolutional kernel propagated bottom can be used by multiple CWs, generating multiple elements in the corresponding output channel.

We define a single operation of SA as starting from the first data input to the upper-left PE to the last element of the last row's CW input to the bottom-right PE. For a complete operation in Figure 2.5, by tracking the data flow of the last row of the SA, it can be found that after three cycles of the start cycle, the first element of the last row's CW reaches the bottom-left PE, and after another three cycles, the data reaches the bottom-right PE. After an additional 27 cycles, the last element of the CW reaches the bottom-right PE. In total, it takes 33 cycles for the data flow to be completed. Equation 2.5 shows a general method to calculate the ideal cycles for one SA operation. Here, we only focus on the time of data flow, without considering the calculation time and the time to obtain the outputs. Because the time of data flow is fixed while the calculation time and the time to obtain the outputs vary with different hardware implementations. For example, the OS SA stated in [16] requires additional SR cycles to collect the outputs when the PE at the rightmost corner has finished computation.

$$\text{Time}_{\text{trans}} = \text{SR} + \text{SC} - 2 + \text{CWS} \quad (2.5)$$

WS is a data flow pattern in which the weights are stationary and the inputs and

outputs flow in both directions of the **SA**. Figure 2.6 shows an **SA** constructed based on **WS**. Initially, the weights are pre-loaded into the **PEs** from the left side. Each row of the **SA** corresponds to a convolutional kernel, and each **PE** in each row stores one element of the corresponding convolutional kernel. Figure 2.6 uses a 4×27 array, which can store 4 convolutional kernels simultaneously, and all 27 elements of each convolutional kernel can be pre-loaded into the **PEs**. This enables it can complete the convolution shown in Figure 2.4 without re-loading the weights. Input data continuously flows into the array from the upper edge, and the input of two adjacent columns of data are separated by one cycle. Whenever the input data enters a **PE**, it is multiplied by the pre-loaded weights, and the result of the multiplication is added to the partial sum from the left **PE** to obtain a new partial sum. This process repeats until the right edge **PE** produces an output. However, if the number of columns in the **SA** is not large enough to store all the elements of a kernel simultaneously, the array needs to be used multiple times to calculate the final outputs.

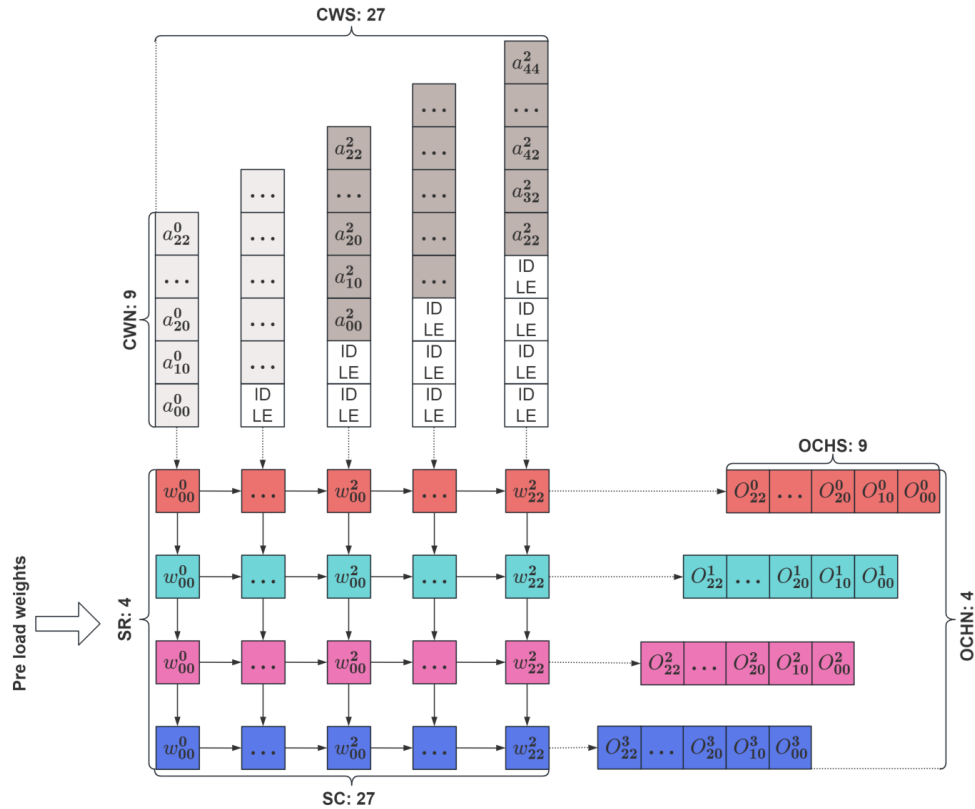


Figure 2.6: Weight stationary dataflow pattern of a systolic array

WS and **OS** have similar data reuse and also face the same scaling problem,

which will not be explained further here.

$$\text{Time}_{\text{trans}} = 2 \cdot \text{SC} + \text{SR} + \text{CWN} - 2 \quad (2.6)$$

As for the ideal time cycles, there are slight differences between **WS** and **OS**. The formula is shown in Equation 2.6. Specifically, it first takes SC cycles to pre-load the weights into **PEs**. Then starting with the upper-left **PE** first receiving input data, it then takes SC-1 cycles for the upper-right **PE** to receive input for the first time, and another SR-1 cycles for the bottom-right **PE** to receive data. Finally, after CWN cycles, all the **CWs** are processed.

IS is almost exactly the same as **WS**. It just exchanges the operations on weights and inputs. So it will not be discussed in detail here.

2.3 Related work

In general, the related work in this field can be explained from two aspects: software and hardware.

At the software level, it can be divided into software simulators and software design automation tools. Software simulators can help researchers quickly explore different hardware structures and compare the performance of different structures and data flows. N. Bohm Agostini *et al.* proposed a new framework named TFLITE-SOC [18], which integrated SystemC into Tensorflow Lite. By leveraging the hierarchical design capabilities of SystemC, it is possible to quickly model and evaluate the accelerators designed by SystemC. And it can be tested with any neural network compatible with Tensorflow Lite. It can also provide a comprehensive analysis of accelerator occupancy and execution time. Finally, they used this framework to analyze the design space of a **TPU**-like **SA** and provided optimization directions to prove its function. A. Parashar *et al.* proposed Timeloop [19], a tool for evaluating and exploring the architecture design space of **DNN** accelerators. It can concisely describe various hardware topologies and simulate the topology to find the best way for data storage and operation on a specific architecture, which can inspire researchers to study new **DNN** accelerator architectures. A. Samajdr *et al.* proposed a cycle-accurate simulator for a **DNN** accelerator based on an **SA** [16], which provides a detailed description of three typical **SA** dataflow patterns. This tool enables the comparison of performance when using different dataflow patterns and various array sizes to solve the same

problem.

As for the software automation design tools, they can assist researchers in designing hardware architectures to a certain extent, reducing design difficulty and shortening the development period. L. Jia *et al.* proposed a method that used Chisel [20] to build multiple basic hardware module templates and these hardware modules can be used to realize different data flows and computational algorithms by properly configuring [21]. By using this method, different designs of **SAs** can be easily implemented and have good performance. X. Wei *et al.* proposed an analysis model for the performance and resource utilization of hardware accelerators and developed an automatic design space exploration framework [22], which can realize the transformation from program C to a **CNN** implementation using the **SA**. It can generate corresponding accelerators for actual neural networks.

In hardware design, many researchers have adopted novel methods to enhance performance. E. Baek *et al.* proposed a multi-neural network acceleration architecture, which can fully utilize the computational resources and memory bandwidth of the accelerator matching the parallel execution of different tasks across different networks [23]. It can decompose each layer into multiple sub-layers and perform parallel execution through scheduling control. A. Samajdr *et al.* introduced the self-adaptive reconfigurable arrays [24], which consist of a reconfigurable array and a hardware unit capable of determining an optimized configuration. It can not only achieve high mapping efficiency but also increase data reuse significantly. DyHard-DNN [25] is a new **DNN** hardware accelerator proposed by M. Putic *et al.*, where the parameters can be dynamically adjusted during **DNN** execution to optimize performance. The accelerator utilizing this architecture shows significant improvements in both performance and power consumption compared to the best static configuration in six mainstream **DNN** workloads.

There are also some researchers dedicated to using **SAs** to process sparse matrices. Although **SAs** have good effects in processing dense matrices, they are not suitable for processing sparse matrices due to their relatively regular structure and relatively simple control logic. G. Shomron *et al.* proposed an **Simultaneous Multithreading in Systolic Arrays (SMT-SA)** to process operations on sparse matrices [26]. It uses **Simultaneous Multithreading (SMT)** technology to fully utilize the **PEs** that are idle due to the input of zero values during the operation. At the same time, a tiling method to reduce area

overhead is proposed. B. W *et al.* proposed a method named **Group-Structure-Maintained Compression (GSMC)** to process sparse matrices [27]. Combined with this method, a new **Sparsity-supported Dynamic Dataflow (SDD)** [27] is proposed based on the traditional data flow patterns of **SA**. The **SA** using dynamic data flow significantly improves performance compared to the **TPU** structure with only a slight increase in hardware overhead.

In addition to the new hardware architecture, several novel data quantization methods have also been used in implementations. K. Guo *et al.* proposed a programmable and flexible **CNN** accelerator architecture called Angel-Eye [28], which incorporates an 8-bit data quantization method with negligible precision loss. This architecture exhibits significant improvements in performance and power consumption compared to other accelerators implemented on the same platform. Y. Li *et al.* developed a high-performance accelerator based on an **SA** with a weight stationary dataflow pattern and quantization method [29], achieving advanced levels of data throughput per unit power consumption in the research field. M. Mathew *et al.* proposed a method to simplify **CNN** models [30]. Specifically, it proposed a quick and simple method to sparsify a **CNN** model, recovering lost precision through fine-tuning of the sparse network while quantizing data to 8-bit fixed-point numbers to enhance multiplication efficiency, making it highly suitable for the operational requirements of low-power embedded devices.

2.4 Summary

This chapter focuses on presenting the background knowledge related to this project. It begins by introducing the basic knowledge of **CNNs**. Subsequently, it starts to introduce the key structure **SA** that will be used in this thesis. It specifically analyzes the working mechanism of **SAs** in handling fully connected operations and convolutional operations, as well as the typical three data flow patterns of **SAs**. Finally, at the end of this chapter, some of the latest related works in the field of **SA**-based hardware accelerator design are listed, and their advantages and disadvantages are briefly analyzed. In general, this chapter provides abundant background knowledge and cases, which is helpful for having an overall background understanding of this research field.

Chapter 3

Design of the SA-based CNN accelerator

In this chapter, we will specifically explain the design of the **CNN** hardware accelerator based on **OS SA**. **OS** data flow pattern is much more flexible when scaling up, which means we do not have to take into account the relationship between the dimensionality of the **SA** and the number of elements of the **CW**. It consists of the following five sections: Overview of the SA-based CNN accelerator, Design of Configurator, Design of Folding Controller, Design of Data Storage Module, and Design of Data Processing Module.

3.1 Overview of the SA-based CNN accelerator design

The accelerator can be briefly summarised as the following modules: Top Module, Configurator, Folding Controller, Data Processing Module and the corresponding controllers, as well as Data Storage Module.

- **Top Module** is mainly used to connect Configurator, Folding Controller, Data Processing Module, and Data Storage Module.
- **Configurator** can store the parameters that may be used in **CNN** accelerator in advance. It is used to configure the work mode of the accelerator for each layer of the **CNN** model. When it receives a finish signal from Data Processing Module, it will transmit the parameters of the next layer to the Data Processing Module.

- **Folding Controller** is mainly used to control the selection of data from Data Storage Module. It uses several counters to cooperate and simulate a group of **CWs** sliding on the **INFMAP**. According to the data flow pattern, the input data and weight data will be injected from two different directions of the **SA**.
- **Data Processing Module** consists of an arithmetic module based on an **OS SA**, a pooling module, an **Activation Function (ACT)** module, and their corresponding controllers. It is mainly used to process the operations of the convolutional layers, fully connected layers, and pooling layers in the **CNN** model.
- **Data Storage Module** is customized according to the computational characteristics of the convolutional layers and fully connected layers of the selected **CNN** model. It is mainly used to store the **INFMAP** data, the weight data, the bias data, and the temporary output data during the computation process. Compared to the general method of the accelerator using a high-speed bus to communicate with a **CPU** to access external memory [29], the data storage module in this thesis mainly used the on-chip memory. The advantages of this approach are that it does not need to consider the synchronization of different clock domains, and it is easy to implement. This method can also reduce memory access [8]. The disadvantages are also obvious. This approach needs to customize the storage system according to different **CNN** models, which is less general than the former method. Moreover, because of the limited amount of on-chip memory, we are not able to implement a large **CNN** model by this approach.

As shown in Figure 3.1, at the beginning of each layer, Configurator will pass the configuration parameters of the current layer to other controllers. During each data reading, Folding Controller will transmit the selection information of the current **CWs** and convolutional kernels to the input controller. The input controller selects the input data and weight data according to the information and feeds them from the left and upper boundaries of the array respectively for calculation. After the array completes the data calculation, the calculation results will output from the lower boundary of the array and add to the bias corresponding to each convolutional kernel. After that, the final calculation results will be obtained and stored in the pooling registers. At this time, the counters in Folding Controller will change to simulate the sliding of the **CW** and select the next group of **CWs** and corresponding convolutional

kernels. When the calculation results stored in the pooling registers meet certain conditions, the pooling controller will control the pooling units to read data from the registers for pooling operations. The pooling results are sent to the **ACT** units for nonlinear activation and the output controller will push the activation results to the output memory.

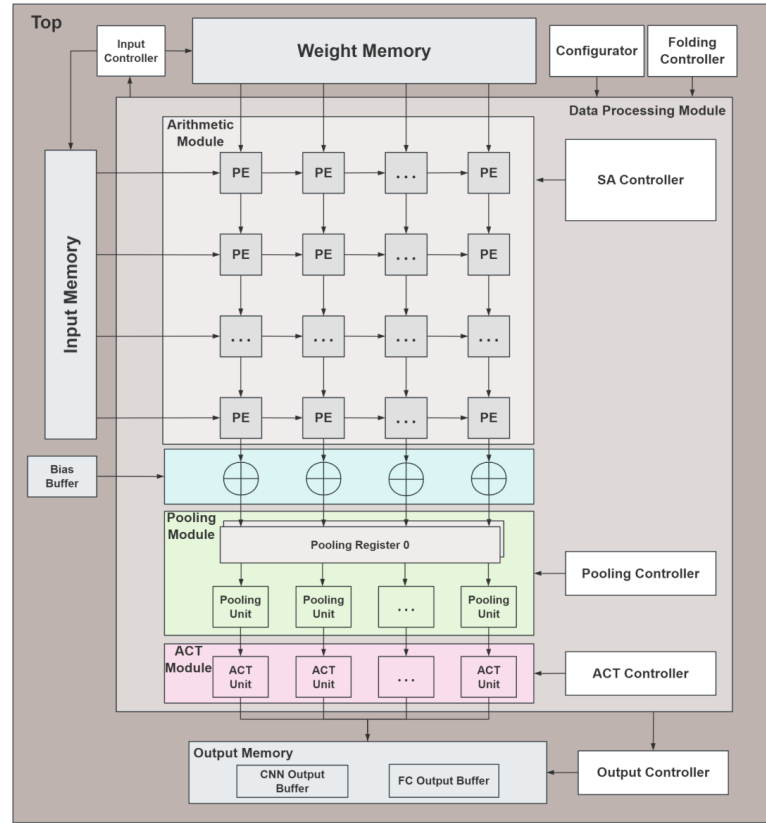


Figure 3.1: Overview of the hardware architecture

The reason why we place the activation function module after the pooling module is mainly because, the amount of data is significantly reduced after the pooling operations, which leads to a corresponding reduction in nonlinear activation operations and can effectively improve the operating efficiency in hardware platforms. At the same time, the pooling operations in this design are mainly max-pooling, and common nonlinear activation functions such as Sigmoid and Tanh can be considered as monotonically increasing, and ReLU can be considered as monotonically increasing within the positive number range and always zero within the negative number range. Therefore, performing nonlinear activation operations before or after max-

pooling operations does not affect the results in the inference stage.

For the data flow pattern of the entire accelerator, we have implemented sequential operating between different layers of the CNN model, which means the operation of the next layer will only start after the operation of the current layer is completely completed. However, the various operation steps within one layer are fully parallel.

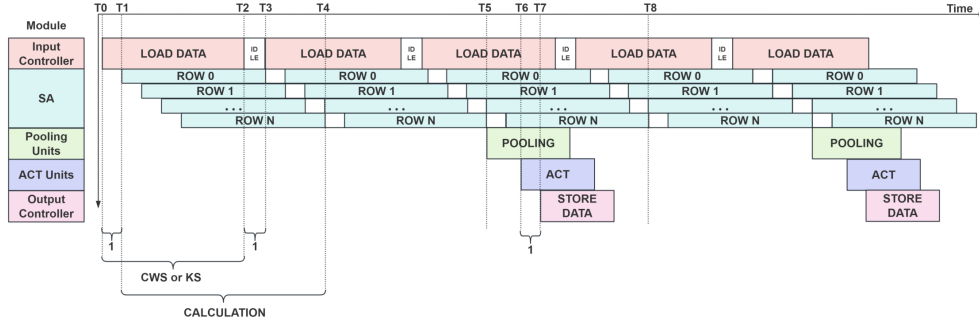


Figure 3.2: Overview of the dataflow timeline

Figure 3.2 shows a simplified example that describes our specific accelerator data flow. A LOAD DATA process represents taking the input data of SR **CWs** and the weight data of SC convolutional kernels from Data Storage Module and feeding them into Data Processing Module gradually. For the selected SR **CWs**, at each cycle, one data is selected from each window. This process repeats for CWS times until all elements of the selected SR **CWs** have been selected. The weights in the convolutional kernels are performed similarly. A CALCULATION process is to send the weight data and input data from the LOAD DATA process into the **SA** to complete the calculation.

After multiple CALCULATION processes, the data stored in the registers is subjected to pooling and activation operations, and the final results will be stored in the output buffers. The reason why we perform a POOLING process after multiple CALCULATION processes is that the data obtained from one CALCULATION process cannot fill a pooling window. In this figure, we use two LOAD DATA processes and the corresponding CALCULATION processes, but the exact number of times is related to the number of rows in the **SA**, the dimensions of the pooling window, and the pooling stride.

Finally, there are a small number of IDLE processes in Figure 3.2. Each of them only consists of 1 cycle. This 1 idle cycle is mainly used to clear the

registers in the **PEs** in preparation for the next CALCULATION process.

At the same time, we also mark some important time points. T_0 represents the time when the first selected SR input data and SC weight data are output from Data Storage Module. T_1 is the time when the data fetched at time T_0 enters the **SA**. Generally, T_1 is one cycle later than T_0 . T_2 is the time when the last elements of the currently selected **CWs** and convolutional kernels are fetched. There are CWS cycles between T_2 and T_0 . T_3 is similar to T_0 , but it is the first time that data from newly selected **CWs** and convolutional kernels are fed into Data Processing Module. T_4 is the time to complete the calculations of all the data input from the first LOAD DATA process. From T_1 to T_4 is a CALCULATION process. T_5 is the time to fill up the pooling window with the calculation results and start the pooling operations. T_6 is the time to get the first group of pooling results and start the activation operations. T_7 is the time to get the first group of activation results and start to store the results in the output memory.

3.2 Design of Configurator

Configurator is the core control unit of the entire **CNN** hardware accelerator. It records the parameters required for running each layer in the **CNN** model. As the brain of the **CNN** accelerator, it guides the communication between Data Processing Module and Data Storage Module, as well as how each specific step in Data Processing Module should be executed.

We use a **Finite State Machine (FSM)** to represent each layer of the **CNN** model in Configurator. This means that each state in the **FSM** represents a layer in the **CNN**. When Data Storage Module is initialized, the **FSM** enters the first layer and passes the parameters of the first layer to Data Processing Module. Subsequently, every time Data Processing Module completes all the operations of the current layer, it sends a finish signal to Configurator to tell it that it can switch to the next layer.

To ensure the performance of the accelerator, we sacrifice area in exchange for higher speed. We pre-load the parameters of each layer in a look-up table. When Configurator switches to a certain layer, it directly reads the parameters of the corresponding layer from the look-up table without any calculation. The design difficulty of this Configurator is how to select parameters that can directly reflect the characteristics of the convolutional layers or fully connected

layers and can correctly guide Data Processing Module to perform various steps such as convolution calculation, pooling, and activation.

3.3 Design of Folding Controller

As we stated in Chapter 2, the resources on a chip are limited, and we cannot arbitrarily expand the dimensions of an SA. However, the dimensions of the INFMAP in a CNN can be set very large. Generally, the SA cannot process all the convolutional kernels and all the CWs on the INFMAP at one time. A feasible scaling method is to design an SA with appropriate dimensions, select a part of convolutional kernels and CWs each time, and complete the calculations of a convolutional layer multiple times. We divide the calculation data into several blocks and term the method of using the SA to process these data blocks multiple times as folding [16].

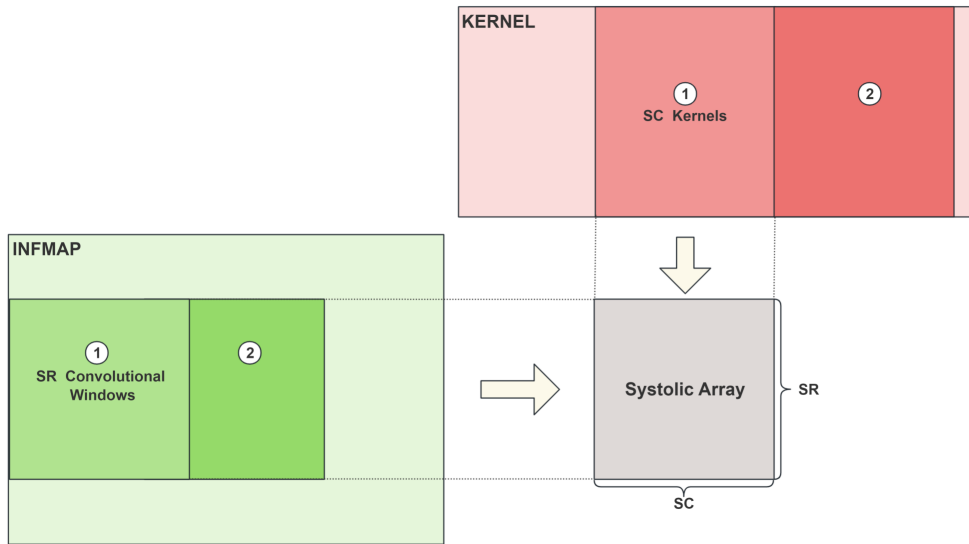


Figure 3.3: Process of the foldings

Figure 3.3 shows a folding process. For an $SR \times SC$ SA, it can process SR CWs and SC convolutional kernels simultaneously. Figure 3.4 shows a sliding process of a group of CWs on the INFMAP. Combining Figure 3.3 and Figure 3.4, the working mode of Folding Controller is as follows. Initially, Folding Controller first selects SR CWs in the upper-left corner of the INFMAP, which are represented by different colors. At the same time, Folding Controller selects SC unused convolutional kernels. Folding Controller

fixes the convolutional kernels and continuously inputs the elements of the convolutional kernels from the columns of the **SA** while the group of **CW** starts to slide on the **INFMAP**. It first slides horizontally according to Step 1 in Figure 3.4, selects different input data, and keeps feeding these data into the array from the rows. When this group of **CW** slides to the right boundary of the **INFMAP**, it slides down once according to Step 2 in Figure 3.4 and then continues to slide horizontally. When this group of **CWs** completely covers the **INFMAP** by sliding, it means that the input data used to compute all the elements to the corresponding SC output channels have been completely input into the array. When the computation is complete, the **CWs** return to the initial position, and then SC more new convolutional kernels are selected according to Step 2 in Figure 3.3, and the above steps repeat until all the elements of all the output channels are computed.

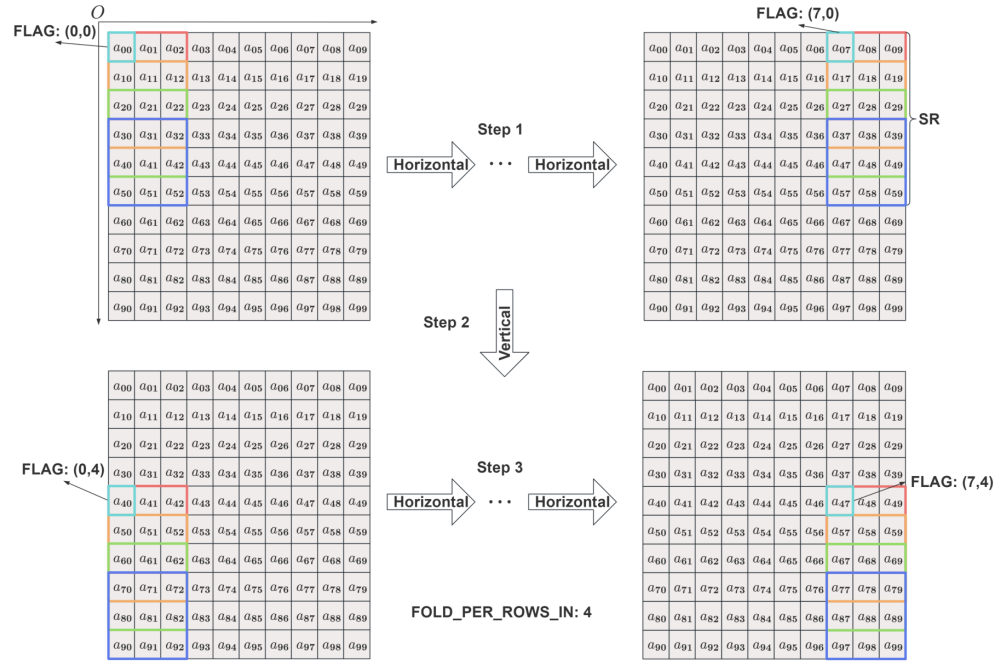


Figure 3.4: Sliding process on convolutional windows

In summary, there are two types of folding, folding on the **CWs** and folding on the convolutional kernels. Each slide of the **CWs**, either horizontally or vertically, is the former one. Each time Folding Controller selects unused convolutional kernels is the latter one. The total number of foldings can be calculated by multiplying the two types of folding.

3.4 Design of Data Storage Module

Data Storage Module can be functionally divided into input memory for storing the initial **INFMAP**, weight memory and bias memory for storing the weights and bias of each layer of the **CNN** model, and output memory for storing the computation results of Data Processing Module. On the other hand, we can structurally divide Data Storage Module into two kinds of memory, one based on the dimensions of **FMAPs** and the other based on the dimensions of the **SA**. The input memory and a part of the output memory can be regarded as the former type, while weight memory, bias memory, and another part of the output memory can be regarded as the latter type. Because some of the memory blocks need to be customized according to the dimensions of the **FMAPs**, this design method is less general and requires a deeper understanding of the convolutional process. In this section, we will introduce these memory blocks according to their function. The specific hardware implementations will be introduced in the next chapter.

3.4.1 Design of the input memory

The input memory is mainly used to store the initial **INFMAP**. It can be formed by combining multiple **Random Access Memorys (RAMs)** or **Read Only Memorys (ROMs)** according to the actual requirements. If we need to get data from the outside, for example, to collect data from a camera, we can use **RAMs** to form the memory, which needs an additional logic for writing data. Otherwise, we can simply use multiple **ROMs** to form the memory. Once the input memory is initialized at the beginning, Data Processing Module can read data from it. In this design, we mainly focus on how Data Processing Module reads data from the input memory, so we use **ROMs** for an explanation.

We assume that the first layer of the **CNN** must be a convolutional layer. For the data reading method of the convolutional layer, we can decompose it into two steps as follows:

1. **The selection of **CWs**.** For an $SR \times SC$ **SA**, SR **CWs** will be selected for each folding on the **CWs**.
2. **The selection of elements in the SR **CWs**.** Data Processing Module will select one element from every **CW** in this group at each cycle, for a total of SR elements as input data.

Therefore, how to read SR elements in one cycle in parallel is the main difficulty of reading data to the SA in convolutional layers.

In this design, we divide the input memory into multiple memory blocks according to the channels of the INFMAP, which means each memory block stores one channel of the INFMAP. At the same time, each memory block is composed of multiple ROM blocks. Each ROM stores one row of one channel of the INFMAP. When reading data, a memory block is determined first. Then, in that memory block, multiple ROM blocks will be selected simultaneously and these ROMs share one address. Each cycle all of these selected ROMs will output one data and be concatenated together and these data will be sent to Data Storage Module. In this way, parallel reading can be achieved.

3.4.2 Design of the weight memory

The weight memory and bias memory can be classified and explained as one category. These two are similar in structure and operation mechanism. These are mainly used to store the convolutional kernels of each layer in the CNN model and the bias corresponding to each convolutional kernel respectively. At the same time, we do not need to design the weight memory and bias memory separately for the convolutional layers and the fully connected layers. As we mentioned in Chapter 2, the fully connected layer can be regarded as a special case of the convolutional layer. For the fully connected layer, a group of weights for each output neuron can be regarded as a convolutional kernel. We will mainly use the weight memory as an example to explain this category.

For the weight memory, we can certainly construct it in a similar way as the input memory. However, this construction method is not very suitable for the operation mechanism of the weight memory. Introducing this method will make the problem more complicated. Although the data reading method of the weight memory is similar to the input memory, which can be decomposed into two steps, namely the selection of convolutional kernels and the selection of elements in the convolutional kernels, it still has several differences. For the input memory, there are some overlapping elements in a group of CWs selected each time, and there are also some overlapping elements between two slides. For the weight memory, multiple convolutional kernels selected each time are independent of each other, and the convolutional kernels selected every two times are also independent of each other. Therefore, we can expand a convolutional kernel from a multi-dimensional matrix to a one-dimensional

vector and store it continuously in a **ROM**. The same method can be used to store the remaining convolutional kernels one by one in this **ROM**.

Considering the requirement of parallel reading, we expand one **ROM** into multiple **ROMs**. The number of **ROM** blocks is consistent with the number of columns of the **SA**. The convolutional kernels are stored in these **ROMs** in sequence after being unfolded. During each data reading, all **ROMs** in the weight memory will be selected and share one address. In this way, multiple data can be read out at one time and input into each column of the **SA** respectively.

3.4.3 Design of the output memory

The output memory is mainly used to store the output results after completing operations of each layer in the **CNN**. For all layers except the first layer, the current layer will read the results of the previous layer from the output memory and use them as input data. Therefore, the output memory must have both the functions of reading and writing data, so we need to use multiple **RAMs** to implement it. With the current data flow pattern, the three steps of data reading, data calculation, and data writing are performed in parallel, rather than one step beginning after the previous one completes. In order to avoid conflicts between read and write operations, as well as the possibility of newly written data overwriting the data that has not been used, we decide to divide it into two blocks as shown in Figure 3.5. Data Processing Module reads the output data of the previous layer from one block as input and writes the output data of this layer into another one.

The current output memory design is not a general memory structure like the weight memory that can not only be used to store both the weights of convolutional layers but also those of fully connected layers. We have designed two different memory structures to store the output results of convolutional layers and fully connected layers respectively. Although this method is not very efficient in terms of storage space utilization, it can reduce the complexity of control logic to a certain extent.

The **CNN** output buffer is mainly used to store the output results of the convolution layer and the **Full Connected (FC)** output buffer is mainly used to store the output results of the fully connected layer. The former is similar in structure and data reading process to the input memory, which is constructed

based on the dimensions of the **INFMAP**, while the latter is more similar in structure to the weight memory. When output data is written each time, multiple memory blocks of the **CNN** output buffer will be selected at the same time, which means that multiple output channels are selected. One **RAM** block is selected in each selected memory block and these **RAMs** share one address. In this way, multiple data can be written into the **CNN** output buffer in parallel at one cycle. For the **FC** output buffer, its writing and reading processes are relatively simple. In each cycle, all the **RAMs** in the **FC** output buffer will be selected at the same time and share one address so that multiple data will be written at the same time. For the reading process, one **RAM** in the **FC** output buffer is selected and one data will be read from that **RAM** in each cycle.

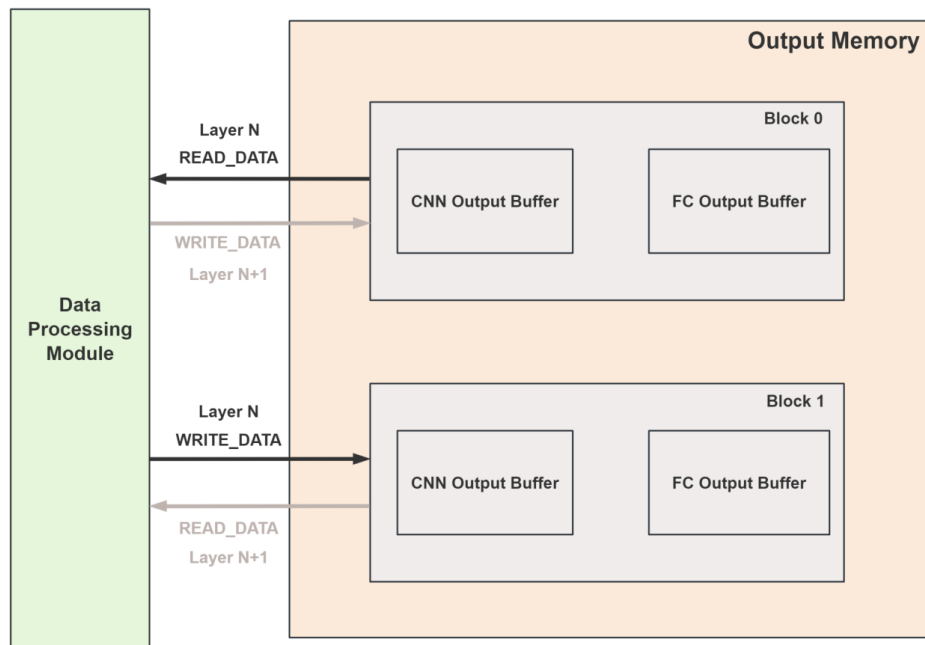


Figure 3.5: Overview of the output memory

3.5 Design of Data Processing Module

Data Processing Module is mainly composed of three modules, namely the arithmetic module based on the **OS SA**, the pooling module, and the activation

function module.

This module performs the operations of the convolutional layers and the fully connected layers in a similar way. We can think that there is only one **CW** in the fully connected layer, so only the **PEs** in the first row of the array participate in the work. At the same time, the calculation results of the array are directly subjected to nonlinear activation operations without pooling operations.

3.5.1 Arithmetic module

The core of the arithmetic module is an **OS SA**. It is mainly used to handle convolution calculations and fully connected calculations in the **CNN** model. In this design, we set the number of rows and columns of the **SA** to be equal, which means **SR** is equal to **SC**.

PE is the core component for constructing an **SA**. Through different designs and different arrangements of **PEs**, **SA** has multiple structures and data flow patterns, which enables the **SA** can be used to handle different problems. One **PE** in the array needs to perform one multiplication operation and one addition operation each time data is input. Therefore, the core of the **PE** in this array is a multiplier and an adder. The selection of the multiplier can be made according to the actual design requirements. In this design, we choose a pipelined multiplier for multiplication operations. This multiplier can handle the multiplication of signed fixed-point numbers. It performs a shift-and-add operation in each cycle. After the same number of cycles as the data bit width **DW**, the multiplication result can be obtained. At the same time, each **PE** also has a data bypass. The data bypasses of all **PEs** in a column will be connected as the output path of the output results. When the calculation is completed, the output result of each **PE** will be selected in turn and output through this output path.

The **SA** controller can also be regarded as a sub-module of the arithmetic module. Its main role is to control the output results of the **SA** to be correctly stored in the registers. Due to the **OS** data flow pattern, the data input of two adjacent **PEs** differs by one cycle. Therefore, there is also a difference in cycles in data output. In terms of timing, it shows a stepped output.

The core of this controller is one or more counters. When the **PE** at the upper-left corner of the array completes the calculation of a **CW** and obtains an output

result, the counter starts to count. Under different count values, the controller will select the results output from the bottom of the SA and store them in the corresponding positions of a temporary register. When this register is full, all the data will be stored in the pooling registers and wait for pooling operations.

3.5.2 Pooling module

The pooling module is mainly used for pooling the output results of the arithmetic module in convolutional layers. It is mainly composed of several registers, several pooling units, and a pooling controller. The number of pooling registers is related to the pooling strides and the dimensions of pooling windows. The number of pooling units is consistent with the number of columns of the SA. The pooling controller is mainly used to control the data transfer from the pooling registers to the pooling units.

In this design, the pooling module is mainly used to handle the max-pooling operations and can only handle the case where the pooling stride is equal to the dimensions of the pooling window. When both of them are one, only one pooling register is involved in the work. In this case, it is equivalent to not performing pooling operations and directly passing the calculation results of the SA to the activation function module. When both of them are equal to two, two pooling registers will be enabled to store the results of the SA. When these two registers are full, pooling operations will begin. As the pooling stride and the dimensions of the pooling window continue to increase, the number of pooling registers that need to be used also increases.

During one pooling operation, each output channel has a pooling window that selects a set of data from the pooling registers and gradually inputs these data into the pooling unit. Each input data will be compared with the larger value of the previous cycle until all the data of the pooling window participate in the comparison to obtain the result of the max-pooling operation.

For a fully connected layer in the CNN module, it will not be connected to the pooling layer under normal circumstances, and the outputs of the SA will be directly connected to the activation function module without the pooling module.

3.5.3 Activation function module

The activation function module is a relatively simple module. It is composed of several **ACT** units. Each **ACT** unit is responsible for performing nonlinear activation operations on the results of one output channel. When the pooling unit completes a pooling operation, it will output the current pooling result and an enable signal to the corresponding **ACT** unit. If the activation unit recognizes this signal, it will perform an activation operation on the received pooling result. After completing the activation operation, the **ACT** unit will also send an enable signal to the output controller. When the output controller recognizes this signal, the result of the **ACT** unit will be written to the corresponding position in the output memory under the control of the controller.

3.6 Summary

This chapter focuses on explaining the design of the **SA**-based **CNN** hardware accelerator. The chapter begins by providing an overview of the design and introducing the module composition of the accelerator. Then, we briefly describe the parallel timing design and its basic operating mechanism. Subsequently, it elaborates in detail on the working mechanism of each part in the accelerator and its communication mechanism with adjacent modules. In general, this chapter can well present our design ideas and is helpful for an overall understanding of our designed accelerator.

Chapter 4

Implementation

In this chapter, we will specifically explain how to implement those modules introduced in Chapter 3 and several optimizations on the hardware level. The neural network accelerator in this thesis is implemented through pure programmable logic.

4.1 Implementation of Configurator and Folding Controller

Configurator not only contains basic parameters such as the dimensions of the **INFMAP** and the dimensions of the convolutional kernel but also contains other more complex parameters, especially some judgment conditions in Folding Controller. Table 4.1 lists the basic parameters that we can configure in the Configurator.

Several parameters need to be explained in detail. **CNN_SIG** indicates the method to read the input data. Since the way the convolutional layers read input data is different from that of the fully connected layers in this design, we set this parameter to select the method for the different layers. The meaning of **KERNEL_DIM** is also slightly different in the convolutional layers and the fully connected layers. In the convolutional layers, it represents the dimensions of the convolutional kernel. However, in the fully connected layers, it is equal to the number of columns of the **SA**. This difference is mainly caused by the different operating characteristics of the convolutional layers and the fully connected layers. **PW_PER_PERIOD** represents the number of pooling operations (the number of pooling windows) required to process

the data of one output channel stored in the pooling registers during one POOLING process.

Table 4.1: Parameters and definitions in Configurator

Parameters	Definition
LAYER_INDEX	The index of the layer
CNN_SIG	Method of reading input data
POOLING_EN	Whether to perform pooling operations
KERNEL_DIM	Dimensions of one kernel matrix
KERNEL_NUM	Number of kernels
KERNEL_ELEMENT	Number of elements in a kernel including all channels
IN_CHANNEL	Number of input channels
STRIDE	Strides in convolution
INFMAP_DIM	Dimensions of one INFMAP matrix
OFMAP_DIM	Dimensions of one OFMAP matrix
POOLING_DIM	Dimensions of one PFMAT matrix
POOLING_KERNEL_DIM	Dimensions of a pooling window
POOLING_WINDOW_NUM	Total number of pooling windows
POOLING_STRIDE	Stride in pooling
PW_PER_PERIOD	Number of pooing windows in one POOLING process

Several parameters that are mainly related to folding are further defined upon the basic parameters listed in table 4.1. Therefore, we will clarify these parameters in combination with the implementation of Folding Controller.

Folding Controller is composed of a set of counters. FOLD_ROWS_CNT counts the number of foldings on the **CWs**. FOLD_COLS_CNT records the number of foldings on the convolutional kernels. When the selected SR **CWs** slide once on the **INFMAP**, FOLD_ROWS_CNT is incremented by one. When the new SC convolutional kernels are selected, FOLD_COLS_CNT is incremented by one as well. For FOLD_ROWS_CNT, it can be decomposed into FOLD_ROWS_CNT_X and FOLD_ROWS_CNT_Y. We take the upper-left corner element of the first **CW** among the SR **CWs** as FLAG, which is the same as the FLAG shown in Figure 3.4. The two counters represent the horizontal coordinate and vertical coordinate of the FLAG. When this group of **CWs** slides horizontally once, FOLD_ROWS_CNT_X will plus STRIDE. When this group of **CWs** slides vertically once, FOLD_ROWS_CNT_Y will plus STRIDE $\times$ SR. This difference in the horizontal coordinate and vertical coordinate is caused by the fact that one group of the **CWs** is composed of

consecutive SR **CWs** in the vertical direction.

After introducing the detailed implementation of Folding Controller, we introduce five new parameters as follows:

- **FOLD_ROWS**: This parameter represents the total number of foldings on **CWs**. When FOLD_ROWS_CNT is equal to FOLD_ROWS, the counter will be set to zero on the next cycle. Meanwhile, FOLD_ROWS_CNT_X and FOLD_ROWS_CNT_Y will also be set to zero.
- **FOLD_COLS**: This parameter represents the total number of foldings on convolutional kernels. FOLD_COLS_CNT will be increased by one when FOLD_ROWS_CNT is equal to FOLD_ROWS. When both of the counters are equal to their maximum values, this means all input data and weight in this layer have been sent into the **SA**.
- **FOLD_PER_ROWS_IN**: This parameter is mainly used to control the selection of the **CWs** on the **INFMAP**. It is numerically equal to the maximum value of FOLD_ROWS_CNT_Y. In other words, this parameter is used to calibrate the upper limit of the vertical coordinates of the first element of the first **CW** of the set of **CWs** on the **INFMAP**.
- **FOLD_PER_COLS_IN**: This parameter is mainly used to control the output channel selection during data output. For an $SR \times SC$ **SA**, SC convolutional kernels are selected for operation each time. This means that each column of the array will input a convolutional kernel. In other words, each column of the array corresponds to an output channel. Similarly, we take the first output channel of each selected group of output channels as FLAG. Initially, FLAG is 0. Each time Folding Controller selects new convolutional kernels, FLAG will plus SC. This parameter is equal to the value of FLAG after the last convolutional kernel selection is performed. In summary, this parameter calibrates the upper limit of the first output channel of the selected group of output channels.
- **POOLING_WINDOW_LAST_PERIOD**: This parameter is mainly used to control the arrangement of data in an output channel during data output. For the generation process of a **PFMAP**, each time the POOLING process is performed, pooling operations are carried out on a group of pooling windows to obtain multiple elements of the

PFMAP. Suppose that each POOLING process generates two elements in one output channel. These two elements are arranged in the vertical direction to form a set of pooling outputs. The next set of pooling outputs will be arranged in the horizontal direction. When reaching the right boundary, the next set of pooling outputs will be arranged vertically. The arrangement of pooling outputs on the **PFMAP** is similar to the selection method of **CWs** on the **INFMAP**. Similarly, we take the first element of each group of pooling outputs as FLAG. Each time it is arranged vertically, FLAG will plus the number of elements in one output channel generated by one POOLING process. This parameter is equivalent to the value of FLAG after the last vertical arrangement. In summary, this parameter is used to calibrate the upper limit of the vertical coordinate of the first element of the group of pooling outputs in one output channel on the **PFMAP**.

4.2 Implementation of Data Storage Module

In the previous, we have already described the design methods of Data Storage Module. We will focus on describing the hardware architectures and the data communication methods of each memory.

4.2.1 Hardware architecture of input memory and data reading process

Figure 4.1 shows the structure of the input memory. We use multiple memory blocks to form the input memory. Each memory block is used to store a channel of the **INFMAP**. For one memory block, it contains **INFMAP_DIM ROMs** and each of the **ROMs** stores one row of the **INFMAP**. The advantage of this design is that Data Processing Module can pick the data in different **ROMs** under the same address, reading multiple data in parallel at the same time. It can also accurately simulate the sliding of the **CW** on the **INFMAP**.

The method of Data Processing Module reads the data from the input memory as follows. We still define the first element of the first **CW** among the SR **CWs** taken out at a time as FLAG. We use **FLAG_ROM_SELECT** to represent the **ROM** where FLAG is stored. This means that according to this signal and **STRIDE**, the **ROM** where the first element of each **CW** in this group of the **CWs** is stored can be indexed. We use **ROM_SELECT_CW_BASE [SR-1:0]**

to represent them. This means that SR ROMs are selected at the same time when reading the data. Similarly, ADDR_BASE represents the address of FLAG in the ROM.

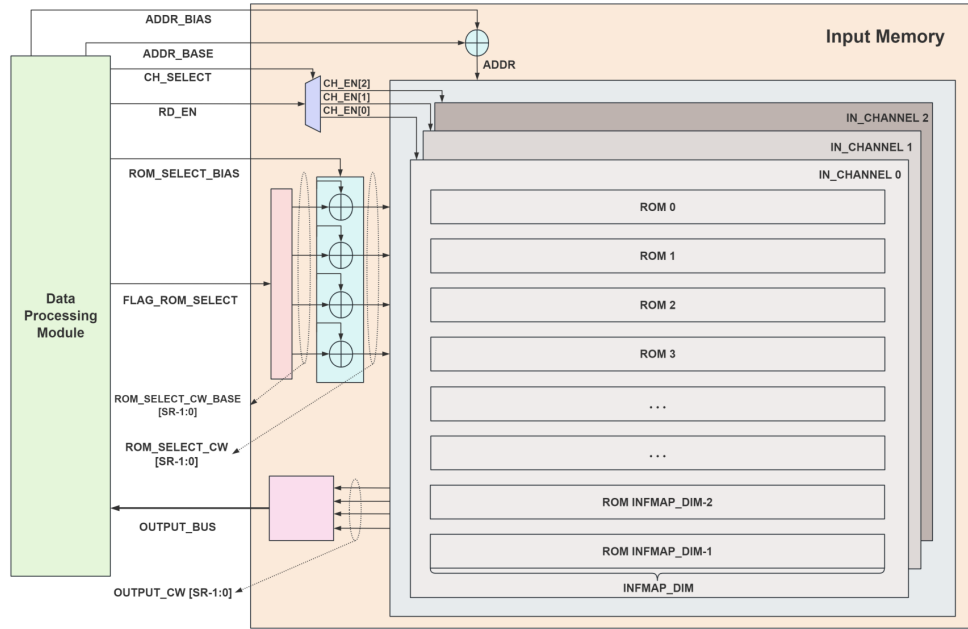


Figure 4.1: Hardware architecture of the input memory

After calibrating the ROM where the first element of each CW in this group is stored and the address in the corresponding ROM, it is relatively simple to select other elements in the CW. The range of ROM_SELECT_BIAS is between 0 and KERNEL_DIM-1, and the range of ADDR_BIAS is also between 0 and KERNEL_DIM-1. The first element of each CW is located as the base ROM and base address of the corresponding CW. By adding an offset to the base ROM and base address, any element in the CW can be indexed. ROM_SELECT_CW [SR-1:0] and ADDR can represent the actual position of any element in the group of the CWs. When reading elements from a CW, it follows the principle of first reading horizontally within a channel and then reading vertically. After one channel is read completely, it will switch to the next channel and perform the above steps again. This process will repeat until the data of all channels are selected. After that, according to the control of Folding Controller, FLAG_ROM_SELECT and ADDR_BASE will change. This process can be regarded as the sliding of the CWs.

From the perspective of the output port, we used OUTPUT_CW [SR-1:0] to

represent the elements output by the selected **ROMs** in one cycle. These SR output elements are concatenated together through a module and then sent to Data Processing Module through OUTPUT_BUS.

4.2.2 Hardware architecture of weight memory and data reading process

In this section, we will mainly use the weight memory as an example to explain the hardware architecture. The mechanisms of the weight memory and bias memory can be regarded as the same.

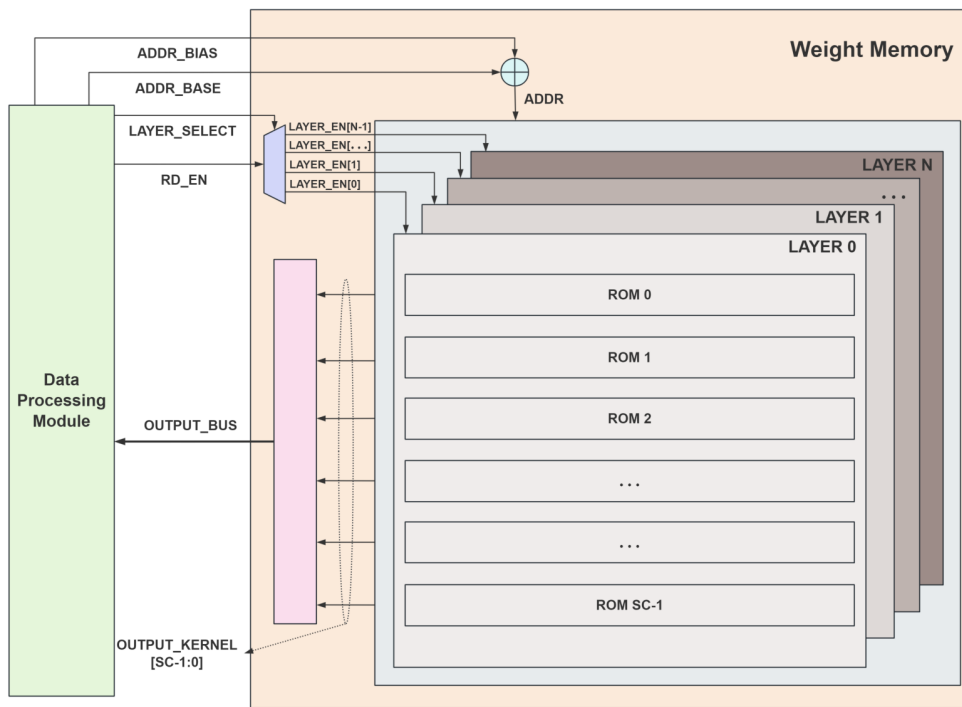


Figure 4.2: Hardware architecture of the weight memory

Figure 4.2 shows the process of Data Processing Module reading data from the weight memory. We use multiple memory blocks to construct the weight memory. Each memory block is used to store all the convolutional kernels of one layer. For a memory block, it contains SC **ROMs**, which is equal to the columns of the **SA**. Since there may be a situation where KERNEL_NUM cannot be divided evenly by SC, both Equation 4.1 and 4.2 need to be used to calculate the size of the **ROM**. Generally, for SC **ROMs**, there are $\text{KERNEL_NUM} \% \text{SC}$ **ROMs** whose sizes are calculated by Equation 4.1, and

the others are calculated by Equation 4.2. Convolutional kernels are stored first in the vertical direction and then in the horizontal direction in a memory block. For example, the first convolutional kernel stored in **ROM** 0 is the first of all convolutional kernels, and the second convolutional kernel stored in **ROM** 0 is the $(SC+1)^{th}$ of all convolutional kernels.

$$ROM_SIZE = \lceil \frac{KERNEL_NUM}{SC} \rceil \cdot KERNEL_ELEMENT \quad (4.1)$$

$$ROM_SIZE = \lfloor \frac{KERNEL_NUM}{SC} \rfloor \cdot KERNEL_ELEMENT \quad (4.2)$$

The workflow of Data Processing Module reads the data from the weight memory as follows. Configurator will select the memory block according to the LAYER_INDEX. In the weight memory, all **ROMs** in a memory block will be generally selected to output data. ADDR_BASE represents the address of the first element of the convolutional kernel selected by the current folding in the **ROM**. ADDR_BIAS is the offset of other elements relative to the first element of the convolutional kernel. With ADDR_BIAS plus ADDR_BASE, we can obtain the address ADDR of any element in the currently selected convolutional kernel. ADDR_BASE will change as the folding changes. By adding KERNEL_ELEMENT to the previous ADDR_BASE, we can obtain the current folding's ADDR_BASE. We repeat the above process to access other elements of the new convolutional kernel.

From the perspective of the output port, we used OUTPUT_KERNEL[SC-1:0] to represent the elements output by the selected **ROMs** in one cycle. These SC output elements are concatenated together through a module and then sent to Data Processing Module through OUTPUT_BUS.

The bias memory and the weight memory have slight differences. In the weight memory, a convolutional kernel contains multiple weight data, so ADDR_BIAS is introduced to represent the offset of other elements relative to the first element of the convolutional kernel. In the bias memory, we can regard that a convolutional kernel has only one data, so ADDR_BIAS is equal to zero and KERNEL_ELEMENT is equal to one.

4.2.3 Hardware architectures of output memory and data communication process

This section mainly introduces the specific hardware architectures and corresponding reading and writing processes of the **CNN** output buffer and the **FC** output buffer.

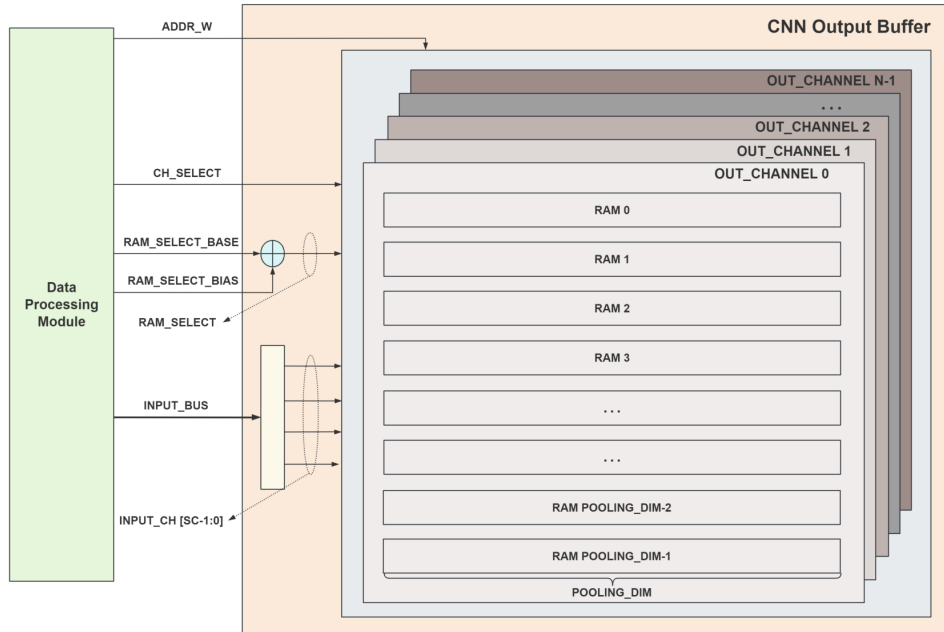


Figure 4.3: Hardware architecture of the CNN output buffer

Figure 4.3 shows the structure of the **CNN** output buffer and the signals related to the writing process. The **CNN** output buffer is composed of multiple memory blocks, and each memory block is used to store one output channel of the **FMAP**. For a memory block, it is composed of **POOLING_DIM RAMs**, and each **RAM** can store **POOLING_DIM** elements. The data reading mechanism of the **CNN** output buffer can be considered the same as that of the input memory, so it will not be explained again here. We will combine the data output pattern of the Data Processing Module to specifically explain how to write data into the **CNN** output buffer.

When Data Processing Module performs the **POOLING** process, the output memory will select the **CNN** output buffer to receive data. At this time, Data Processing Module will output **SC** pooling results in each cycle. **CH_SELECT** represents the selection of the **SC** output channel. Specifically, **INPUT_BUS**

will be split into $INPUT_CH[SC - 1:0]$ through a logic module. These SC pooling results will be written into the SC consecutive memory blocks selected by CH_SELECT .

We only focus on one output channel now. For one POOLING process, multiple pooling windows are processed sequentially in one output channel, and each pooling window will obtain a pooling output. RAM_SELECT_BASE represents the **RAM** where the first pooling output in this group of pooling outputs will be written, and RAM_SELECT_BIAS is the offset of other pooling outputs relative to the first one. RAM_SELECT is equal to the sum of RAM_SELECT_BASE and RAM_SELECT_BIAS , which can be used to index the **RAM** where any pooling output in this group will be written. $ADDR_W$ represents the address in the corresponding **RAM**. For multiple data in one output channel, they are sequentially written into different **RAMs** in the vertical direction. When Data Processing Module tries to perform the next POOLING process, $ADDR_W$ will be incremented. When $ADDR_W$ is equal to $POOLING_DIM - 1$, $ADDR_W$ will be set to 0 and RAM_SELECT_BIAS will also increase accordingly before the Data Processing Module executes the next POOLING process. This process repeats continuously until all pooling results are obtained.

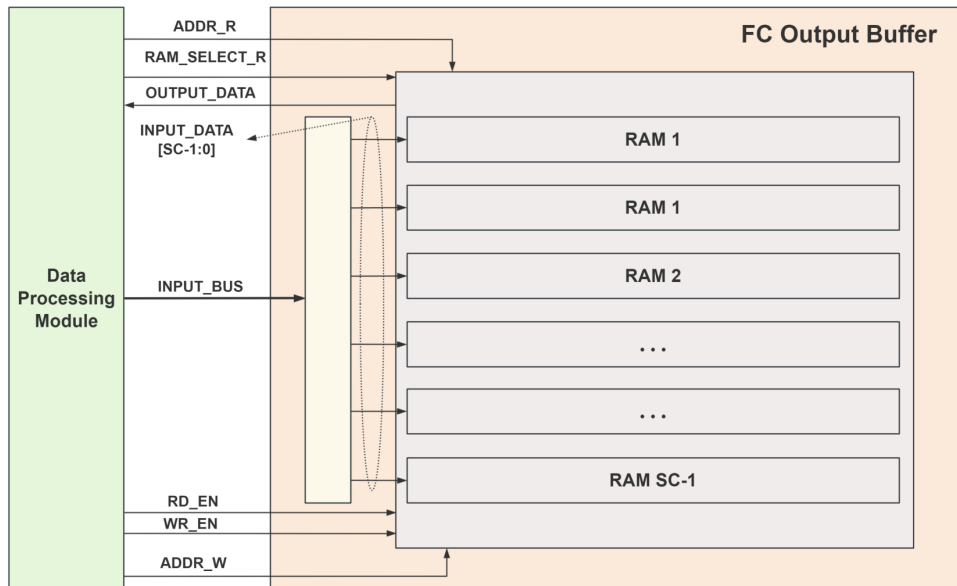


Figure 4.4: Hardware architecture of the FC output buffer

We do not consider the **ACT** process after the POOLING process. This is

because the **ACT** process only transforms the values of the pooling outputs and does not affect the data output pattern of Data Processing Module.

As for the **FC** output buffer, it is composed of SC **RAMs** as shown in Figure 4.4. For the read operations, the **FCs** output buffer can be simply regarded as a two-dimensional storage array. In each cycle, one data is read and input into Data Processing Module according to the vertical coordinate **RAM_SELECT_R** and horizontal coordinate **ADDR_R**. The reading operation follows the rule of reading first in the vertical direction and then in the horizontal direction. When the data is read, **ADDR_R** is kept unchanged and **RAM_SELECT_R** is incremented first. When **RAM_SELECT_R** is equal to **SC-1**, **RAM_SELECT_R** will return to 0 and **ADDR_R** is increased by one in the next cycle. This process repeats continuously until all data is read.

For the write operations, Data Processing Module will output SC data in each cycle. This means there will always be SC data that need to be written to the **FC** output buffer in each cycle. The SC **RAMs** in the **FC** output buffer are generally selected simultaneously during the write operations. **ADDR_W** represents the address where data is written into the **RAMs**. Each time Data Processing Module outputs new data, **ADDR_W** is incremented and these new data are written into the new address of each **RAMs**.

4.3 Implementation of Data Processing Module

This section will focus on introducing the hardware implementation of each module in Data Processing Module and some timing information.

4.3.1 Hardware architecture of arithmetic module

The arithmetic unit undertakes most of the calculations and logical operations of Data Processing Module. In the previous chapter, we have already introduced the basic design idea of the arithmetic module. We will further explain the problems encountered in the actual implementation process and explain its workflow in combination with the actual hardware structure.

Figure 4.5 shows the structure of a **PE**. It receives the input data and the enable

signal from the left boundary, receives the weight data from the top boundary, and outputs the calculation result from the bottom. When the input data and the weight are input into a **PE**, on the one hand, they will be immediately sent to the multiplier for multiplication operation. On the other hand, they will be temporarily stored in the registers for one cycle and output to the neighbor **PEs** on the rising edge of the next cycle. The multiplication result of the input and weight will be sent to the adder and added to the partial sum calculated in the previous cycle to obtain the partial sum of the current cycle, which is temporarily stored in Psum Reg, waiting to be added to the multiplication result of the next cycle. This process repeats until all the input data in the **CW** are multiplied by the corresponding weight data in the convolutional kernel and added to the partial sum. At this time, the **PE** will generate a **FINISH_FLAG** signal indicating that an output result has been calculated. The current partial sum will be stored in Output Reg and wait for output.

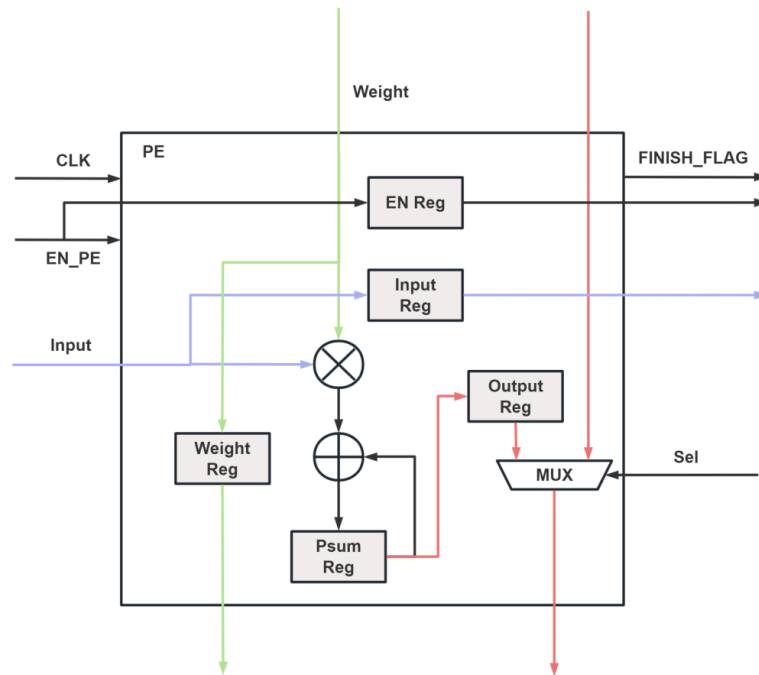


Figure 4.5: Hardware architecture of one processing unit in the systolic array

In our design, it is not necessary to wait until the calculations of all **PEs** in a column are completed before starting to output results. If a **PE** is located in the first row of the **SA**, when it generates the **FINISH_FLAG** signal, it indicates the results of this column will be output step by step. Specifically,

at first, the SEL signals of this column are equal to zero, which means each PE selects the output of the upper PE as the output of the current PE. After the upper-level controller recognizes that the PE in the first row has generated the FINISH_FLAG signal, the SEL signals of each PE in this column will gradually change to one from top to bottom. This means that the calculation results of the PEs in this column will be gradually output from top to bottom.

$$\text{Time}_{\text{cal}} = \text{DW} + \text{KERNEL_ELEMET} + 3 \quad (4.3)$$

$$\text{Time}_{\text{total}} = \text{SR} + \text{SC} + \text{DW} + \text{KERNEL_ELEMET} + 1 \quad (4.4)$$

It takes three cycles from the time the PE in the first row generates the FINISH_FLAG signal to the time the result of this PE is stored in a temporary register. These 3 cycles are fixed and independent of the dimensions of the SA. We can calculate the number of cycles from the time the first input enters a PE to the time the computation result generated by the PE is stored in the temporary register according to Equation 4.3. Moreover, combining Equation 2.5 and 4.3, the total cycles of a CALCULATION process can be calculated.

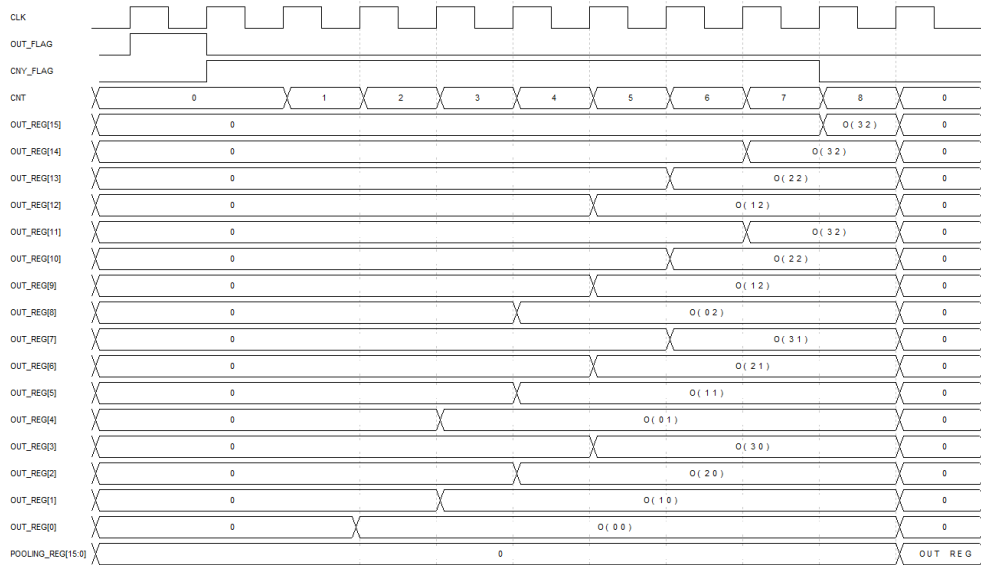


Figure 4.6: Output timing diagram of the systolic array

Figure 4.6 is a data output timing diagram of a 4×4 SA under the control of the SA controller during one CALCULATION process. After the PE in the upper-left corner of the array generates FINISH_FLAG, which is OUT_FLAG in Figure 4.6, the SA controller sets CNT_FLAG to one. When CNT_FLAG is high, the counter will start to count in the next cycle. OUT_REG[15:0] is

used to store the outputs of the **SA**, and it can store a total of 16 outputs. The outputs are gradually stored in OUT_REG[15:0] under the control of CNT. Among them, $O(i, j)$ represents the output data generated by the **PE** in row i and column j . When CNT is equal to $SR + SC - 1$, CNT_FLAG will be set to 0 in the next cycle. When CNT_FLAG is low, in the next clock rising edge, CNT will be cleared to 0, and the data of OUT_REG[15:0] will be stored in POOLING_REG[15:0]. Then OUT_REG[15:0] will be cleared to start to prepare to receive the outputs of the next CALCULATION process.

This **SA** controller centered on a counter can handle most cases, but some additional cases require extra explanation. This is because there is only one idle cycle between two adjacent CALCULATION processes. In the actual timing sequence, at the start of the next CALCULATION process, the previous CALCULATION process has not been completed yet, which may lead to multiple output results competing for one output path. According to the relationship between the number of elements in a **CW** in the current layer and the dimensions of the array, it can be divided into three cases:

1. **KERNEL_ELEMENT+1 > SR+SC-2**. In this case, using one counter can control the output results to be correctly stored in the registers. When the first group of the **CWs** is used as input and the **PE** in the upper-left corner has completed the calculation and is ready to output the result, we take this moment as the benchmark. On the one hand, after $KERNEL_ELEMENT + 1$ cycles, this **PE** generates the first output of the next group of the **CWs**. On the other hand, after $SR + SC - 2$ cycles, the **PE** in the bottom-right corner generates the last calculation result of the first group of the **CWs**. If the time required by the former is greater than that required by the latter, there will be no situation in the array where two different groups of the **CWs** are used as input and the calculation results are output at the same time.
2. **SC-1 < KERNEL_ELEMENT+1 <= SR+SC-2**. In this case, there will be a situation in the array where two different groups of the **CWs** are used as input and the calculation results are output at the same time. This requires two independent counters to control the outputs of different sets of the **CWs** to store in different registers respectively. Figure 4.7 shows a critical situation. At the current moment, the **PE** in the upper-left corner accepts the last element of the first window in the current group of the **CWs** and calculates the first output of the current group of the **CWs**. The **PE** in the bottom-right corner accepts the last elements of the last **CW**

in the previous group of the **CWs** and calculates the last output of the previous group of the **CWs**. The former output is stored in the register under the control of CNT2, while the latter output is stored in another register under the control of CNT1.

3. **KERNEL_ELEMENT+1** ≤ **SC-1**. In this case, there may be two **PE** in the same column that complete the calculation and are ready to output the calculation results at the same time. However, there is only one output path, so these two output results will compete for this one output path. Our design currently cannot handle this situation. This situation is relatively rare in actual **CNN** models because the number of elements in a **CW** is usually greater than the number of rows of the **SA**. If our accelerator encounters this situation, we can only handle this situation by adding additional idle cycles between two **CALCULATION** processes.

The above three cases are theoretical analyses. In actual hardware implementations, the critical condition for distinguishing case 1 and case 2 is $\text{KERNEL_ELEMENT} = \text{SR} + \text{SC}$. This means when KERNEL_ELEMENT is equal to or less than $\text{SR} + \text{SC}$, the double-counter strategy will be adopted. This is an adjustment made to match the signal timing.

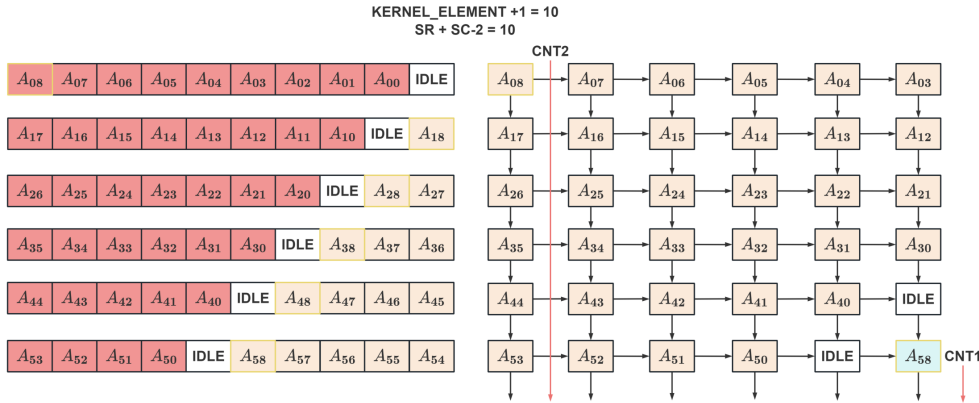


Figure 4.7: The critical situation of two sets of consecutive convolutional windows producing outputs simultaneously in the array

4.3.2 Hardware architecture of pooling module

Figure 4.8 shows a max-pooling operation with $\text{POOLING_STRIDE} = 2$ and $\text{POOLING_DIM} = 2$ in a 4×4 **SA**. The results of the first **CALCULATION**

process will be stored in POOLING_REG1 [15:0]. However, the pooling windows are not filled at this moment, so the pooling operations cannot be performed immediately. It needs to wait for the next calculation results of the SA to be stored in POOLING_REG2 [15:0]. When POOLING_REG2 [15:0] is filled with the elements, the pooling module starts to perform pooling operations. In Figure 4.8, each output channel has two pooling windows to be processed, so the pooling module needs to perform two pooling operations.

In a POOLING process, the data of all pooling registers will be gradually input into the pooling units. The core logic of the pooling controller is still counters: CNT_PL_WINDOW, CNT_PL_WINDOW_X, and CNT_PL_WINDOW_Y. Taking the red output channel in Figure 4.8 as an example, CNT_PL_WINDOW represents the index of the first element of the pooling window selected in this channel, such as O_{00}^0 in Step 1 and O_{20}^0 in Step 2. For other colored output channels, we just need to add the corresponding offset of each output channel to the counter. CNT_PL_WINDOW_X is an offset based on CNT_PL_WINDOW and represents the position of the selected element relative to the selected element of CNT_PL_WINDOW in the horizontal direction. CNT_PL_WINDOW_Y is used to represent the selection of the registers. CNT_PL_WINDOW_X and CNT_PL_WINDOW_Y can be regarded as the horizontal and vertical coordinates in the coordinate system with the element selected by CNT_PL_WINDOW as the coordinate origin. When a max-pooling operation starts, CNT_PL_WINDOW will first select the coordinate origin of this pooling window. Then CNT_PL_WINDOW_X will increment by one in each cycle, reading the data of the pooling window in the horizontal direction first. When CNT_PL_WINDOW_X is equal to POOLING_KERNEL_DIM-1, CNT_PL_WINDOW_X will be cleared to zero and CNT_PL_WINDOW_Y will increment by one in the next cycle. Repeating this process until both CNT_PL_WINDOW_X and CNT_PL_WINDOW_Y are all equal to POOLING_KERNEL_DIM-1. At the same time, CNT_PL_WINDOW will increase by POOLING_STRIDE to select a new pooling window. The data input into the pooling units will be compared with the larger value obtained from the previous cycle. Until all elements of a pooling window participate in the comparison, the final value is the result of the current max-pooling operation. However, due to this data reading mode, the current pooling module can only handle the case where POOLING_STRIDE is equal to POOLING_KERNEL_DIM.

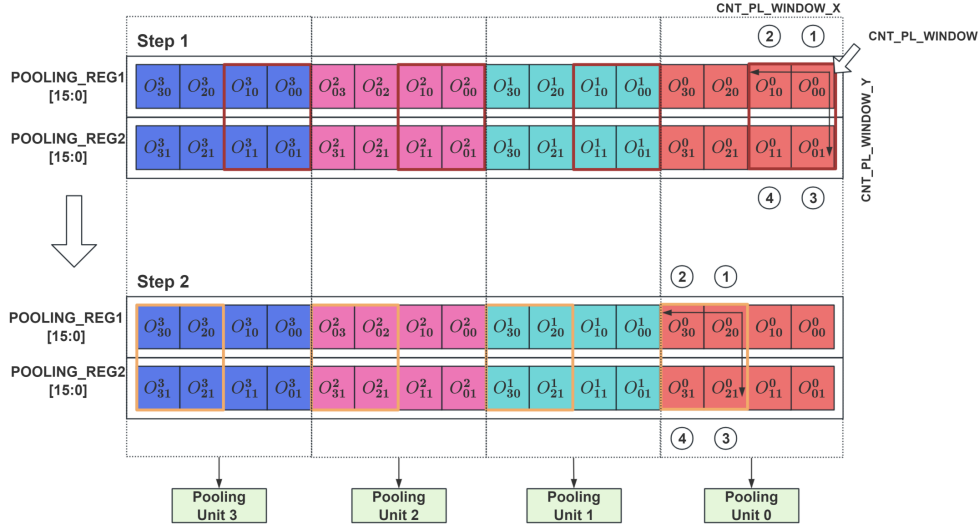


Figure 4.8: Operation process of the pooling units

4.3.3 Hardware implementation of activation function module

In the current design, we use a simple ReLU function for nonlinear activation. The function is shown in Equation 4.5. This operation only takes one cycle to be completed. Although only the ReLU function is currently implemented in the **ACT** unit, we reserve access to other nonlinear functions. Any nonlinear function that can be regarded as monotonically increasing and takes cycles less than the square of POOLING_KERNEL_DIM to complete can directly replace the ReLU function in the **ACT** unit.

$$f(x) = \begin{cases} x & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases} \quad (4.5)$$

4.4 Summary

Based on the design ideas in Chapter 3, we further explain the detailed hardware design and implementation of each module. In this chapter, we combine figures to introduce the key signals and key logic of each module during operation in detail. In summary, we offer a more in-depth way to understand how each module functions within the overall system.

Chapter 5

Results and Analysis

In this chapter, we deploy the accelerator based on a **OS SA** with SR set to 4, SC set to 4, and a total of 16 **PEs** on Xilinx XC7Z020 to verify the correctness of its function and compare its performance with other advanced hardware platforms.

5.1 Testing dataset and data quantization

In the specific implementation process, we choose LeNet [2] as the neural network for testing. This is mainly due to the fact that its model is relatively small and simple to implement, and it is very representative of **CNNs**. Figure 5.1 shows the specific structure of the LeNet model, which consists of two convolutional layers, two max-pooling layers, and two fully connected layers, respectively. At the same time, we use the MNIST [2] database for training and testing. The MNIST database has a total of 70000 28×28 grayscale images. Each pixel in the image is an integer between 0 and 255. Among them, 60000 images are used as training pictures to train the **CNN** model, and the remaining 10000 images are testing pictures used to test the generalization ability of the obtained **CNN** model after training.

We use a laptop with an Intel i5-11300H **CPU** and 16GB of memory to train the **CNN** model. During the training process, we first convert the 28×28 picture into a 32×32 picture by padding zero on the boundaries. After that, the data matrix representing the picture is normalized to a tensor between zero and one and sent into the model for training. After the training is completed, we store the weight parameters and bias parameters of each layer in different text files respectively. These parameters are all 32-bit floating-

point numbers, but our accelerator cannot directly process 32-bit floating-point numbers. Moreover, performing floating-point number calculations on the **FPGA** is more complicated and will significantly increase power consumption and reduce performance. It is not a very good choice. Therefore, before storing the parameters into the **FPGA**, we need to perform data quantization on the floating-point numbers. In order to meet the low-power requirements of mobile devices, we use a 1-bit unsigned integer to represent one pixel in the input image and a 16-bit signed integer to represent one weight data or bias data, which means the first bit is the sign bit and the remaining 15 bits are the numerical bits.

Specifically, we perform binarization processing on the input image. After the data matrix representing the picture is normalized to a tensor between zero and one, each element in the tensor is between zero and one. Zero means that the current pixel appears pure black on the image, and one means the current pixel appears pure white on the image. The color of pixels with values between zero and one is between pure black and pure white. Here we simply think pixels that are not pure black on the image are handwritten strokes. This means the pixels with the current value of zero are unchanged, while the pixels with current values other than zero are all changed to one so that we get an **INFMAP** matrix containing only data zero and one as shown in Figure 5.2. Therefore, in the **FPGA**, we can use a 1-bit unsigned integer to represent any pixel in the picture. At the same time, with the **CNN** model unchanged, we use this **CNN** model to identify the original test dataset and the test dataset with the binarization processing respectively. The experimental results show that the accuracy rate of the **CNN** in predicting the former dataset is 98.95%, and the accuracy rate in predicting the latter dataset is 98.3%. It can be found that there is little change in accuracy when using the **CNN** model to recognize the test set with the binarization processing. This indicates that this binarization processing method is feasible. In specific data processing, the threshold for binarization processing can be flexibly selected according to actual requirements.

For the parameters, we multiply each parameter by 2^{15} and obtain the corresponding integer through rounding. These integers can be represented by 16-bit signed integers in the **FPGA**. At the same time, in the **FPGA**, we perform an arithmetic right shift of 15 bits on each data before the ReLU calculation of each layer to balance the amplification of the parameters. The above multiplication and shifting operations are both linear changes and will not affect the relative size of the data. Although this method loses recognition

accuracy to a certain extent, it can well convert the original floating-point number operation into an integer operation. It also avoids the problem that a large number of multiplication and addition operations in the **FPGA** cause the bit width of the final calculation result to exceed the bit width of the output memory.

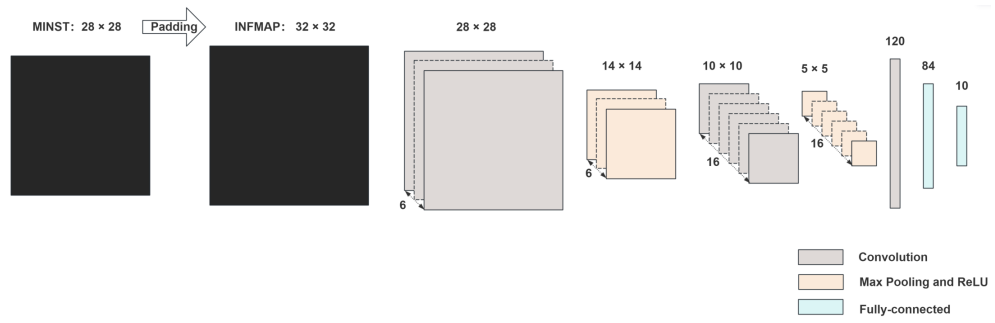


Figure 5.1: LeNet model

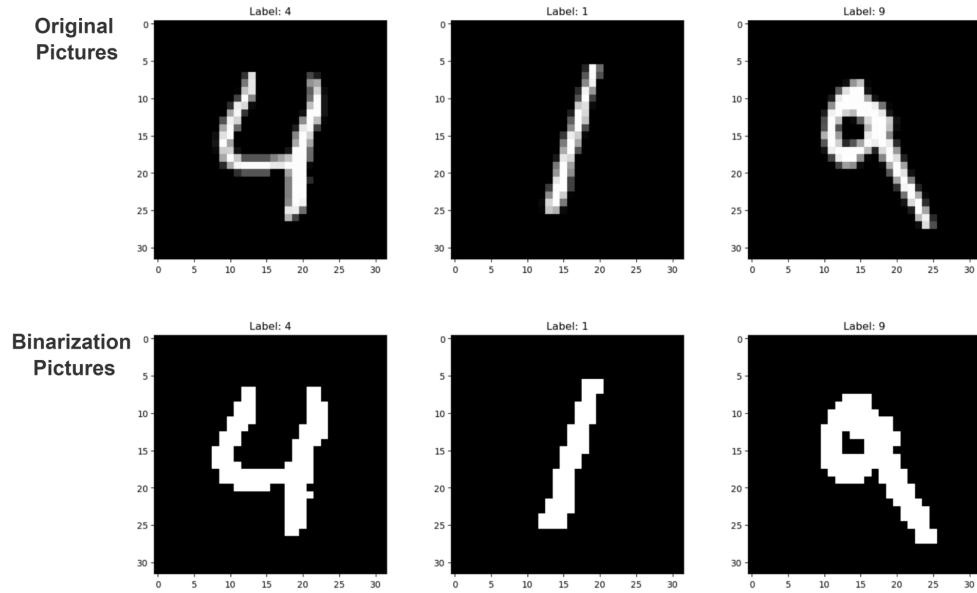


Figure 5.2: Comparison of the dataset before and after binarization

According to the current accelerator architecture and the selected neural network, we have configured the parameters in Configurator as shown in Table 5.1. Each element of the array represents the parameters of the corresponding layer. ‘N’ indicates that this parameter has no practical effect in the current layer. At the same time, since the double-counter strategy in Section 4.3.1 is

not used under the current circumstance, so we only use one counter to control the output of the **SA** calculation results to reduce the use of hardware resources.

Table 5.1: Parameters and the corresponding values in Configurator

Parameters	Value
LAYER_INDEX	[1, 2, 3, 4, 5]
CNN_SIG	[1, 1, 1, 0, 0]
POOLING_EN	[1, 1, 0, 0, 0]
KERNEL_DIM	[5, 5, 5, 4, 4]
KERNEL_NUM	[6, 16, 120, 84, 10]
KERNEL_ELEMENT	[25, 150, 400, 120, 84]
IN_CHANNEL	[1, 6, 16, 1, 1]
STRIDE	[1, 1, 1, N, N]
INFMAP_DIM	[32, 14, 5, 1, 1]
OFMAP_DIM	[28, 10, 1, 1, 1]
POOLING_DIM	[14, 5, N, N, N]
POOLING_KERNEL_DIM	[2, 2, N, N, N]
POOLING_WINDOW_NUM	[196, 25, N, N, N]
POOLING_STRIDE	[2, 2, N, N, N]
PW_PER_PERIOD	[2, 2, N, N, N]
FOLD_ROWS	[195, 29, 0, 0, 0]
FOLD_COLS	[1, 3, 29, 20, 2]
FOLD_PER_ROWS_IN	[24, 8, 0, N, N]
FOLD_PER_COLS_IN	[4, 12, N, N, N]
POOLING_WINDOW_LAST_PERIOD	[12, 4, N, N, N]

5.2 Function validation

We implement this **CNN** model using Python, the embedded processor Cortex-A9, and the **FPGA** Xilinx XC7Z020 according to the above data quantization method. The **CNN** model implemented by Python will be used as a reference to verify the correctness of the calculation results in the **FPGA**. It should be noted that in Python, we do not use 16-bit signed integers to represent the model parameters. However, we ensure that the model parameters and the **INFMAP** matrix are numerically the same as in the other two implementation methods, and the processing methods for the intermediate results obtained from each layer are the same. We first use the test dataset of the MINST to verify the accuracy of the **CNN** model implemented by Python. By performing inference

on 10000 images and comparing them with the actual labels, the final accuracy is 97.65%. It is worth mentioning that the aforementioned accuracy of 98.3% is the inferred accuracy obtained after binarizing the test images. Here, the accuracy of 97.65% refers to the inferred accuracy obtained after binarizing the test images and quantizing the parameters of the **CNN** model. Therefore, we only need to ensure that under the condition of the same **INFMAP** matrix and **CNN** model parameters, the calculation results in the **FPGA** are consistent with the calculation results using Python, which can prove that the function of the **CNN** accelerator deployed on the **FPGA** is correct.

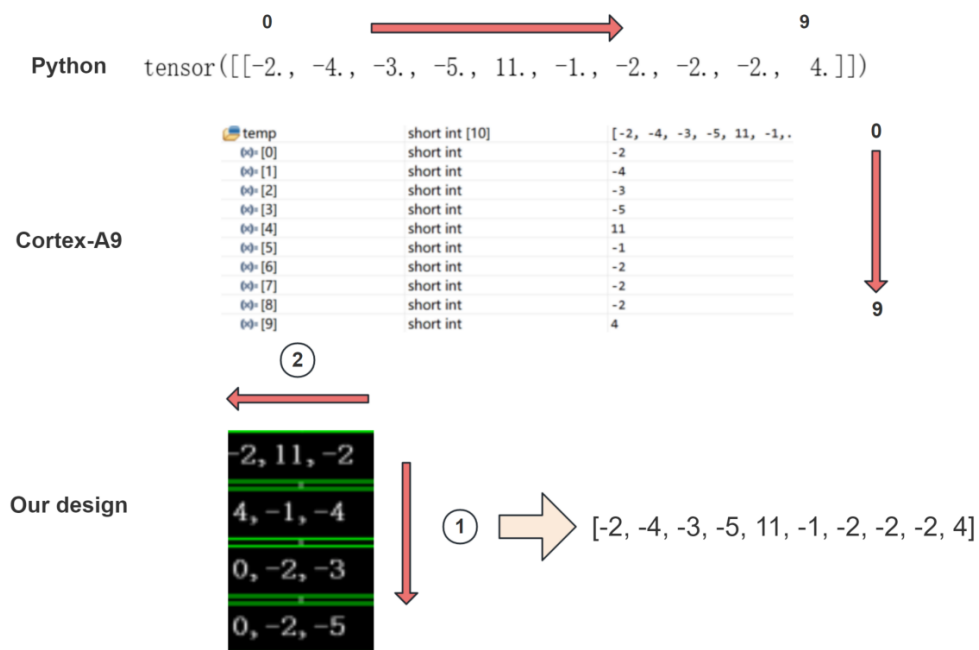


Figure 5.3: Function validation results of our design

We select several different **INFMAPs** and conduct several tests. Figure 5.3 shows a test example. From top to bottom, they are the last layer results of Python, the results in Cortex-A9, and the results of post-implementation simulation in the **FPGA**. It can be seen that the final results of the **CNN** model implemented by three different methods are consistent. At the same time, due to the possible loss of data precision caused by data quantization in the actual calculation process, while comparing the output results of the final layer, we also compared the output results of the intermediate layers to double-check the function of our **CNN** hardware accelerator.

5.3 Performance results

The hardware resource utilization of the accelerator designed in this program is presented in Table 5.2. Our accelerator is not specifically optimized for the **FPGA**. Both Data Processing Module and the corresponding control module are implemented entirely using programmable logic. The synthesis results indicate that a significant number of **Look-Up Tables (LUTs)** are utilized to realize the required logic. Additionally, Data Storage Module is implemented entirely using on-chip memory, which decreases the need to access external memory during data communication. This approach enhances the design performance and reduces the power consumption. However, a notable drawback is that, due to the limited on-chip hardware resources, it is not feasible to deploy very large neural networks. In this design, a substantial amount of **Block Random Access Memorys (BRAMs)** in the **FPGA** is used to store **CNN** parameters.

Table 5.2: Hardware resource consumption

Resource	Used	Available	Utilization %
LUT	41518	53200	78.04
LUTRAM	2064	17400	11.86
FF	20338	106400	19.11
BRAM	62	140	44.29

Table 5.3: Hardware resource consumption of each module in the accelerator

Module	LUT	LUTRAM	FF	BRAM
Configurator	6536	0	170	0
Data Storage Module	7725	2064	3293	62
Data Processing Module	27257	0	16875	0
Total	41518	2064	20338	62

Table 5.3 shows the hardware resource consumption of each module in our design. This table shows that Data Processing Module uses the most **LUTs** and registers to process data, and Data Storage Module uses all the **BRAMs** and uses some of the **LUTs** as memory to store data. Meanwhile, Configurator mainly consists of several **LUTs** and registers to store the configuration parameters of each layer. The reason why the hardware resource consumption of Folding Controller is not shown here is that it is not encapsulated as

a module in the practical code implementation, which is included in Data Processing Module.

Further, Table 5.4 shows the hardware resource consumption in each sub-module of Data Processing Module. The arithmetic module uses the most hardware resources, which are mainly consumed by the 16 PEs in the SA. These PEs use 98.9 % of the LUTs and 97.6% of the registers in the arithmetic module, and the rest hardware resources are used to build the control logic. The ‘Others’ section of this table contains Folding Controller and some other simple circuits used to connect the sub-modules in Data Processing Module.

Table 5.4: Hardware resource consumption of each sub-module in Data Storage Module

Module	LUT	LUTRAM	FF	BRAM
Arithmetic Module	20424	0	14456	0
Pooling Module	957	0	173	0
ACT Module	2829	0	192	0
Others	3047	0	2054	0
Total	27257	0	16875	0

Table 5.5: Performance comparison with other hardware platforms

Device	Data type	Time(ms)	Power(W)	FPS	FPS/Power
Tesla T4 GPU	float32	0.5216	29.79	1917	64.35
CPU in Google Colab	float32	1.2056	\	829	\
i5-11300H	float32	0.6223	\	1607	\
Cortex-A9	float32	93.20	1.28	10.73	8.38
Cortex-A9	int16	92.75	1.28	10.78	8.42
Ours(90MHz)	int16	0.4814	0.381	2077	5451.44

Table 5.5 presents our performance results. We compared the performance of one GPU, two CPUs, and an embedded processor with our design. For the GPU and CPUs, we constructed the CNN model using PyTorch’s default 32-bit floating-point numbers, while for our design and the Cortex-A9, we utilized the data quantization method described in Section 5.1.

Time refers to the time to infer a single image in the MINST. Due to the instability in the time measured for a single operation on the GPU, CPUs, and

Cortex-A9, we select to process one image at a time and calculate the average time for a single operation by recording the execution time across multiple operations. In contrast, the time taken by our design for inferring a single image is relatively fixed. We get the actual cycle numbers for inferring one image from Vivado, which is determined to be 43330 cycles. Multiplying the cycle numbers by the operating frequency of 90 MHz for this design on the **FPGA** yields the inference time of 0.4814 milliseconds per image.

The power consumption of the **GPU** is directly measured using the tool from **NVIDIA Management Library (NVML)**. The power consumption of the **CPU** can not be directly measured at this time. The power estimates of our design and the Cortex-A9 are obtained from Xilinx Vivado after the implementation step. The specific power consumption of our design can be found in Table 5.6. FPS denotes the frames processed per second. It can be calculated by taking the reciprocal of the time taken to infer a single image in seconds.

Table 5.6: Power consumption

Module	Dynamic Power(W)	Utilization %
Clocks	0.085	32
Signals	0.028	11
Logic	0.018	7
BRAM	0.013	5
MMCM	0.122	46
Total	0.266	100

According to Table 5.5, our design achieves the shortest inference time per image, slightly faster than the **GPU** and the two **CPUs**. However, this does not imply that our design outperforms the above **GPU** and **CPUs**. This is because modern **GPUs** and **CPUs** can process multiple images simultaneously. Although the time to process a single image remains unchanged, parallel processing of multiple images significantly enhances system performance. Currently, our accelerator is limited to processing one image at a time. However, when we view it as a computational unit and compare it with the individual computational unit of the **GPU** or **CPU** that processes one image at a time, our design demonstrates a faster inference speed. Moreover, our design significantly outperforms the embedded processor Cortex-A9, achieving a 192 times speed improvement in inference while consuming only one-third of its power. This demonstrates the excellent power efficiency of our design.

5.4 Summary

In this chapter, we provide an introduction to the neural network model and test dataset we have selected, as well as the structure and configuration information of the accelerator deployed on Xilinx XC7Z020. Simultaneously, we adopt an efficient data quantization method to convert the **CNN** parameters from 32-bit floating-point numbers into 16-bit fixed-point integers that can be processed by our accelerator. This method can effectively reduce the bit width and further reduce the hardware resources required for data storage and the computing speed while the inference accuracy of the model hardly changes. Meanwhile, the conversion of the **CNN** parameters from floating-point numbers to fixed-point numbers can also significantly reduce the computational complexity. Then, we verify the function of our design by comparing the calculation results in the **FPGA** with those in Python. Finally, we display various performance parameters of our design, such as hardware resource consumption, power consumption, and the speed of processing one image, and conduct a comparison with other advanced hardware platforms. In general, this chapter mainly focuses on performing a performance evaluation of our design, which helps to gain an in-depth understanding of the design from a performance perspective.

Chapter 6

Conclusions and Future work

In this chapter, we will provide a summary and reflection on this project, and briefly outline our future work plans.

6.1 Conclusions

In this paper, we implement a **CNN** accelerator based on an output stationary systolic array and deploy the accelerator on the **FPGA**, which is verified by processing a real **CNN**. The accelerator is built with pure programmable logic, and we utilized on-chip memory to store the **CNN** model parameters and intermediate computation results. This approach significantly reduces the frequency of accessing external memory, thereby improving performance and decreasing power consumption. Additionally, we employ a specialized data quantization method for the **CNN** platform to reduce the size of the **CNN** model and the computational complexity. Compared to advanced general-purpose **CPUs** and **GPUs**, our design demonstrates a certain advantage in energy efficiency. Moreover, it significantly outperforms the embedded processor Cortex-A9 in both inference speed and power consumption.

The **CNN** accelerator we implemented has largely fulfilled the initial objectives. We convert the convolutional operation into a general matrix multiplication similar to the fully connected operation by expanding the **INFMAP** according to the **CWs** and expanding the convolutional kernels, so that we can build an **SA** that can handle both of these two calculations simultaneously. In the data storage module, we realize the requirement of parallel data reading and writing by sharing an address with multiple memory blocks. At the same time, our designed accelerator data flow enables several

stages such as data loading, data processing, and data storage to run in a pipelined manner within a layer in the **CNN**, greatly reducing idle cycles and improving the operating efficiency of the accelerator. Finally, we select a data quantization method to convert the floating-point numbers in the **CNN** model into integers and apply it in the **FPGA**. It can effectively reduce the network size and increase the operation speed under the acceptable range of precision loss.

6.2 Limitations

Our design also has some limitations. Although we consider this design to be a general architecture, it does not imply that the size of the **SA** can be arbitrarily adjusted, nor does it mean that any type of **CNN** model can be deployed on this accelerator. On the other hand, the data storage module is implemented entirely with on-chip memory. While this design decreases the frequent access to external memory and improves performance to some extent, the limited on-chip hardware resources constrain the deployment of large **CNN** models on accelerators.

6.3 Future work

Our future work will mainly focus on the following three aspects:

1. We plan to develop a generator that defines several mathematical formulas to assist in configuring the parameters in Configurator of our design. This generator only requires several basic parameters from **CNN** model, such as the dimensions of **INFMAPs**, the number of convolutional kernels, and the dimensions of the convolutional kernels, to compute the other necessary parameters required for the accelerator during the actual operation.
2. The timing of this accelerator still has space for further improvement. For the timing path which is not the critical path, we can try to optimize it. Without changing the critical path, we can try to replace some of the sequential logic circuits in these paths with combinational logic circuits. This can reduce the circuit operation cycles without changing the actual operating frequency.
3. We will attempt to design a general data storage module, such as the accelerator obtains data by accessing off-chip memory, and compare it

with the current design that completely uses on-chip memory to form the data storage module. We will analyze the advantages and disadvantages of them and further optimize the current design.

References

- [1] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015. doi: 10.1038/nature14539 [Page 1.]
- [2] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. doi: 10.1109/5.726791 [Pages 1, 4, 8, and 53.]
- [3] N. P. Jouppi, C. Young, N. Patil, and D. Patterson, “A domain-specific architecture for deep neural networks,” *Commun. ACM*, vol. 61, no. 9, p. 50–59, Aug. 2018. doi: 10.1145/3154484 [Page 1.]
- [4] N. P. Jouppi, C. Young, N. Patil *et al.*, “In-datacenter performance analysis of a tensor processing unit,” in *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017. doi: 10.1145/3079856.3080246 pp. 1–12. [Pages 1 and 10.]
- [5] R. Xu, S. Ma, Y. Guo, and D. Li, “A survey of design and optimization for systolic array-based dnn accelerators,” *ACM Comput. Surv.*, vol. 56, no. 1, Aug. 2023. doi: 10.1145/3604802 [Page 2.]
- [6] A. Fuchs and D. Wentzlaff, “The accelerator wall: Limits of chip specialization,” in *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2019. doi: 10.1109/HPCA.2019.00023 pp. 1–14. [Page 2.]
- [7] Kung, “Why systolic architectures?” *Computer*, vol. 15, no. 1, pp. 37–46, 1982. doi: 10.1109/MC.1982.1653825 [Pages 2, 10, 12, and 13.]
- [8] J. Shen, H. Ren, Z. Zhang, J. Wu, W. Pan, and Z. Jiang, “A high-performance systolic array accelerator dedicated for cnn,” in *2019 IEEE 19th International Conference on Communication Technology (ICCT)*,

2019. doi: 10.1109/ICCT46805.2019.8947127 pp. 1200–1204. [Pages 7 and 24.]
- [9] C. Yang, Y. Yang, W. Yang, L. Huang, and Y. Li, “A general-purpose cnn accelerator based on improved systolic array for fpgas,” in *2022 7th International Conference on Integrated Circuits and Microsystems (ICICM)*, 2022. doi: 10.1109/ICICM56102.2022.10011386 pp. 529–533. [Page 8.]
- [10] Y. Wang, G. Wei, and D. Brooks, “Benchmarking tpu, gpu, and CPU platforms for deep learning,” *CoRR*, vol. abs/1907.10701, 2019. [Page 10.]
- [11] Z. Yang, L. Wang, D. Ding, X. Zhang, Y. Deng, S. Li, and Q. Dou, “Systolic array based accelerator and algorithm mapping for deep learning algorithms,” in *Network and Parallel Computing*, F. Zhang, J. Zhai, M. Snir, H. Jin, H. Kasahara, and M. Valero, Eds. Cham: Springer International Publishing, 2018. ISBN 978-3-030-05677-3 pp. 153–158. [Page 10.]
- [12] R. Xu, S. Ma, Y. Wang, and Y. Guo, “Cmsa: Configurable multi-directional systolic array for convolutional neural networks,” in *2020 IEEE 38th International Conference on Computer Design (ICCD)*, 2020. doi: 10.1109/ICCD50377.2020.00089 pp. 494–497. [Page 10.]
- [13] H. Kung, B. McDanel, and S. Q. Zhang, “Packing sparse convolutional neural networks for efficient systolic array implementations: Column combining under joint optimization,” in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS ’19. New York, NY, USA: Association for Computing Machinery, 2019. doi: 10.1145/3297858.3304028. ISBN 9781450362405 p. 821–834. [Page 10.]
- [14] S. Selvam, V. Ganesan, and P. Kumar, “Fuseconv: Fully separable convolutions for fast inference on systolic arrays,” in *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2021. doi: 10.23919/DATE51398.2021.9473985 pp. 651–656. [Page 10.]
- [15] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, “Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural

- networks,” *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017. doi: 10.1109/JSSC.2016.2616357 [Page 16.]
- [16] A. Samajdar, J. M. Joseph, Y. Zhu, P. Whatmough, M. Mattina, and T. Krishna, “A systematic methodology for characterizing scalability of dnn accelerators using scale-sim,” in *2020 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2020. doi: 10.1109/ISPASS48437.2020.00016 pp. 58–68. [Pages 16, 18, 20, and 28.]
- [17] H. Genc, S. Kim, A. Amid *et al.*, “Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration,” in *2021 58th ACM/IEEE Design Automation Conference (DAC)*, 2021. doi: 10.1109/DAC18074.2021.9586216 pp. 769–774. [Page 16.]
- [18] N. Bohm Agostini, S. Dong, E. Karimi, M. Torrents Lapuerta, J. Cano, J. L. Abellán, and D. Kaeli, “Design space exploration of accelerators and end-to-end dnn evaluation with tflite-soc,” in *2020 IEEE 32nd International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*, 2020. doi: 10.1109/SBAC-PAD49847.2020.00013 pp. 10–19. [Page 20.]
- [19] A. Parashar, P. Raina, Y. S. Shao *et al.*, “Timeloop: A systematic approach to dnn accelerator evaluation,” in *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2019. doi: 10.1109/ISPASS.2019.00042 pp. 304–315. [Page 20.]
- [20] J. Bachrach, H. Vo, B. Richards, Y. Lee, A. Waterman, R. Avižienis, J. Wawrzynek, and K. Asanović, “Chisel: Constructing hardware in a scala embedded language,” in *DAC Design Automation Conference 2012*, 2012. doi: 10.1145/2228360.2228584 pp. 1212–1221. [Page 21.]
- [21] L. Jia, L. Lu, X. Wei, and Y. Liang, “Generating systolic array accelerators with reusable blocks,” *IEEE Micro*, vol. 40, pp. 85–92, 2020. [Page 21.]
- [22] X. Wei, C. H. Yu, P. Zhang, Y. Chen, Y. Wang, H. Hu, Y. Liang, and J. Cong, “Automated systolic array architecture synthesis for high throughput cnn inference on fpgas,” in *2017 54th ACM/EDAC/IEEE Design Automation Conference (DAC)*, 2017. doi: 10.1145/3061639.3062207 pp. 1–6. [Page 21.]

- [23] E. Baek, D. Kwon, and J. Kim, “A multi-neural network acceleration architecture,” in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020. doi: 10.1109/ISCA45697.2020.00081 pp. 940–953. [Page 21.]
- [24] A. Samajdar, E. Qin, M. Pellauer, and T. Krishna, “Self adaptive reconfigurable arrays (sara): learning flexible gemm accelerator configuration and mapping-space using ml,” in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, ser. DAC '22. New York, NY, USA: Association for Computing Machinery, 2022. doi: 10.1145/3489517.3530506. ISBN 9781450391429 p. 583–588. [Page 21.]
- [25] M. Putic, A. Buyuktosunoglu, S. Venkataramani, P. Bose, S. Eldridge, and M. Stan, “Dyhard-dnn: Even more dnn acceleration with dynamic hardware reconfiguration,” in *2018 55th ACM/ESDA/IEEE Design Automation Conference (DAC)*. IEEE Press, 2018. doi: 10.1109/DAC.2018.8465873 p. 1–6. [Page 21.]
- [26] G. Shomron, T. Horowitz, and U. Weiser, “Smt-sa: Simultaneous multithreading in systolic arrays,” *IEEE Computer Architecture Letters*, vol. 18, no. 2, pp. 99–102, 2019. doi: 10.1109/LCA.2019.2924007 [Page 21.]
- [27] B. Wang, S. Ma, Z. Liu, L. Huang, Y. Yuan, and Y. Dai, “Sadd: A novel systolic array accelerator with dynamic dataflow for sparse gemm in deep learning,” in *Network and Parallel Computing: 19th IFIP WG 10.3 International Conference, NPC 2022, Jinan, China, September 24–25, 2022, Proceedings*, 2022. doi: 10.1007/978-3-031-21395-3_4 p. 42–53. [Page 22.]
- [28] K. Guo, L. Sui, J. Qiu, J. Yu, J. Wang, S. Yao, S. Han, Y. Wang, and H. Yang, “Angel-eye: A complete design flow for mapping cnn onto embedded fpga,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 1, pp. 35–47, 2018. doi: 10.1109/TCAD.2017.2705069 [Page 22.]
- [29] Y. Li, S. Lu, J. Luo, W. Pang, and H. Liu, “High-performance convolutional neural network accelerator based on systolic arrays and quantization,” in *2019 IEEE 4th International Conference on Signal and Image Processing (ICSIP)*, 2019. doi: 10.1109/SIPROCESS.2019.8868327 pp. 335–339. [Pages 22 and 24.]

- [30] M. Mathew, K. Desappan, P. K. Swami, and S. Nagori, “Sparse, quantized, full frame cnn for low power embedded devices,” in *2017 IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2017. doi: 10.1109/CVPRW.2017.46 pp. 328–336. [Page 22.]

